CMS ACCESS Model API

Operations Manual – Version 0.9.1

Table of Contents

General Guidance

This Operations Manual provides general implementation guidance applicable to all ACCESS Model APIs.

Common Patterns

All ACCESS Model APIs follow consistent asynchronous processing and parameter patterns for ease of implementation. The Alignment API uses an asynchronous request-response pattern to accommodate the processing time required for backend logic. This pattern allows clients to submit requests and subsequently poll for results without maintaining a persistent connection. The pattern uses two sets of operations to support a submit-and-poll workflow:

1. **`$check-eligibility, $align, $unalign, $report-data`**: Submits a request to the API
2. **`$submission-status`**: Polls for the status of a submitted request

Why Asynchronous?

ACCESS operations can take some time to complete, making a synchronous request-response pattern impractical for real-world implementations.

Processing these operations may require additional steps, such as:

- Conducting additional data validation specific to the ACCESS Model
- Processing complex composition structures with nested sections
- Integrating with multiple backend systems to evaluate ACCESS Model requirements
- Applying control group randomization algorithms

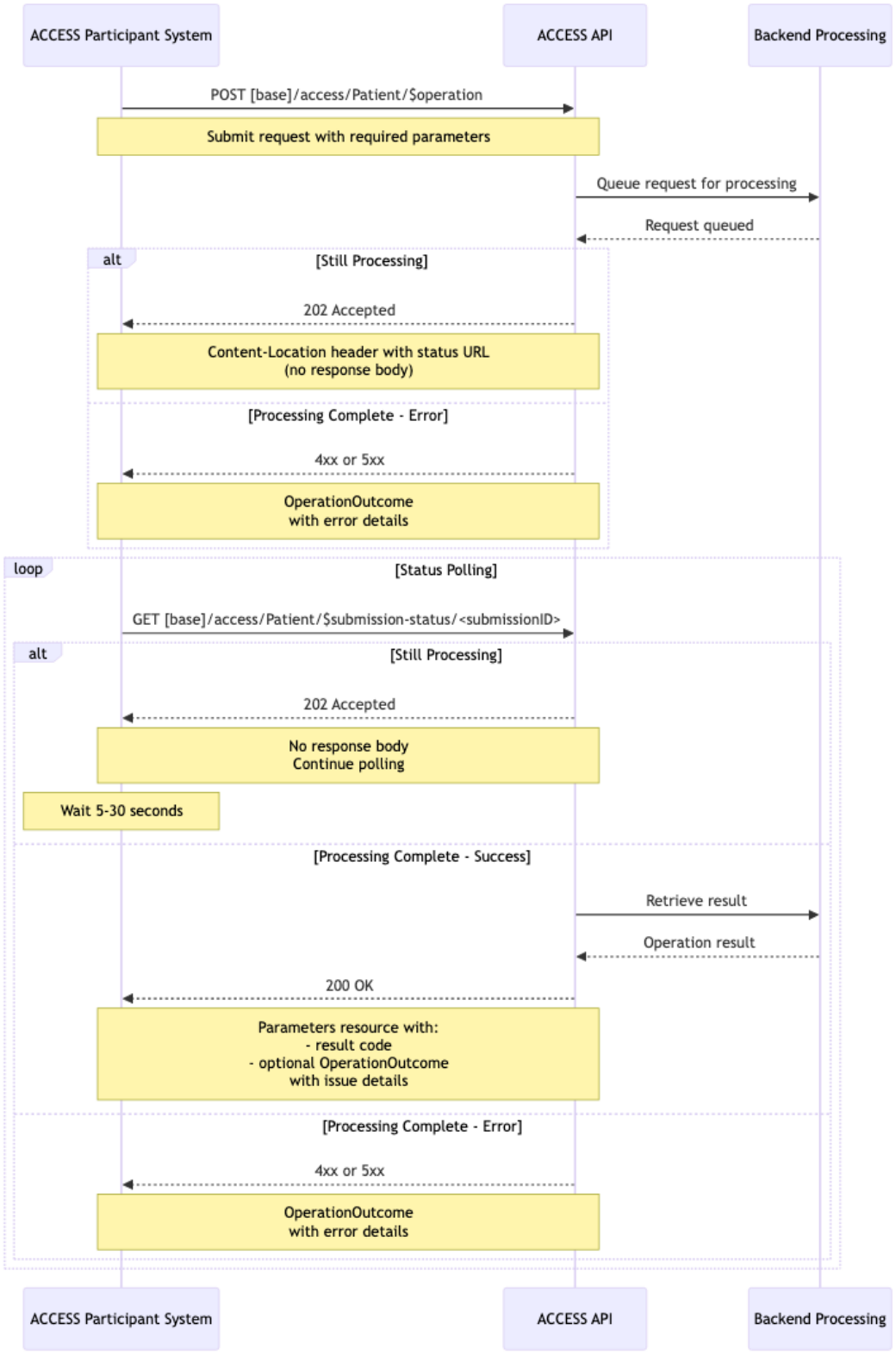
Workflow Overview

The typical workflow consists of the following steps:

1. **Submission**: The client submits a `$check-eligibility`, `$align`, `$unalign`, `$report-data` operation
2. **Acknowledgment**: The server returns `HTTP 202 Accepted` with a `Content-Location` header containing the URL for retrieving submission status
3. **Polling**: The client periodically polls using the `$submission-status` operation through the submission status URL provided
4. **Completion**: When processing completes, the submission status returns `HTTP 200 OK` with the final result

Interaction Pattern:

The following sequence diagram illustrates the general asynchronous pattern used by all ACCESS APIs:


ACCESS Participant System
ACCESS API
Backend Processing
POST [base]/access/Patient/$operation
Submit request with required parameters
Queue request for processing
Request queued
alt
[Still Processing]
202 Accepted
Content-Location header with status URL
(no response body)
[Processing Complete - Error]
4xx or 5xx
OperationOutcome
with error details
loop
[Status Polling]
GET [base]/access/Patient/$submission-status/<submissionID>
alt
[Still Processing]
202 Accepted
No response body
Continue polling
Wait 5-30 seconds
[Processing Complete - Success]
Retrieve result
Operation result
200 OK
Parameters resource with:
- result code
- optional OperationOutcome
with issue details
[Processing Complete - Error]
4xx or 5xx
OperationOutcome
with error details
ACCESS Participant System
ACCESS API
Backend Processing

Required Parameters

All primary operations (`$check-eligibility`, `$align`, `$unalign`, `$report-data`) require these common parameters:

- **`participantID`** - ACCESS participant identifier. This identifier is supplied to the ACCESS participant during the Request for Application (RFA) process and takes the form of "ACCESS<4-digit number>" (ACCESS0001, ACCESS1234)
- **`patientID, name, gender`** – This information conforms with the US Core Patient Profile.
- **`track`** - One of the ACCESS Model tracks (`eCKM, CKM, MSK, BH`)

Some operations may require additional parameters. Please consult the documentation in this guide for information about each specific API.

$submission-status Operation

The `$submission-status` operation retrieves the current status of a previously submitted request. This is a **read-only operation** that uses the HTTP GET method. The URL to use to retrieve the submission status is returned by the primary operation (extracted from the `Content-Location` URL).

Submission Status URL: `GET <submission status URL>`

Output Parameters (when complete):

- **`result`** (CodeableConcept, required): Result code with descriptive text indicating the status. Actual values depend on the API in use.
- **`issues`** (OperationOutcome, optional): This parameter **MAY** be included to provide more detail about the context of the result code.

Example Response from a Primary Operation:

```
HTTP/1.1 202 Accepted
Content-Location: https://[base]/access/Patient/$submission-
status/sub-123456
```

(No response body - processing asynchronously)

Implementation Guidance

Base URL

All primary operations are invoked at the resource level:

```
POST https://[base]/access/[resource]/[$operation]
```

Where `[base]` is the FHIR server base URL provided during participant registration.

Where `[resource]` is Patient.

Where `[$operation]` is the primary dollar sign operation (`$check-eligibility`, `$align`, `$unalign`, `$report-data`)

Content Types

- **Request:** `application/fhir+json`
- **Response:** `application/fhir+json`

Patient Identification

All APIs require a **`patientID`**. Each API recommends the patient's MBI in the Patient resource, whether it is a separate parameter or part of the Data Reporting bundle. To be eligible and align to a CMS ACCESS participant, a patient must be a Medicare beneficiary:

```
{
  "resourceType": "Patient",
  "identifier": [
    {
      "type": {
        "coding": [
          {
            "system": "http://terminology.hl7.org/CodeSystem/v2-
0203",
            "code": "MC"
          }
        ]
      },
      "system": "http://terminology.hl7.org/NamingSystem/cmsMBI",
      "value": "1EG4TE5MK73"
    }
  ],
  "name": [
    {
      "family": "Doe",
      "given": ["John"]
    }
  ],
  "gender" : "male"
}
```

NOTE: The IG will accept any patient identifier and payer identifier so that this IG can be used by other payers looking to align with the ACCESS Model.

Using the Content-Location HTTP Header

ACCESS operations return a `Content-Location` HTTP header containing the URL to check the status of the submission. Clients should:

1. Extract the `Content-Location` header value from the HTTP response
2. Use this URL for subsequent `$submission-status` calls

Example:

```
Content-Location: https://[base]/access/Patient/$submission-
status/sub-123456
```

Polling Strategy

Clients should implement an appropriate polling strategy to balance responsiveness with server load:

- **Initial Poll**: Wait 5-10 seconds after submission before the first status check
- **Subsequent Polls**: Poll every 10-30 seconds while receiving HTTP 202 Accepted
- **Maximum Duration**: Implement a timeout (recommended: 5 minutes) after which the client should stop polling and notify the user; the actual timeout duration should be configured based on operational requirements
- **Exponential Backoff**: Consider implementing exponential backoff to reduce server load for long-running requests

Polling Logic:

```
1. Initiate the ACCESS operation (POST $<operation> with required
parameters)
2. Receive HTTP 202 with Content-Location
3. Wait initial delay (5-10 seconds)
4. While (timeout not exceeded):
   a. Poll status (GET $submission-status)
   b. If HTTP 202: wait and continue loop
   c. If HTTP 200: process result and exit
   d. If HTTP 4xx/5xx: handle error and exit
5. If timeout exceeded: handle as error
```

Interpreting Responses

HTTP 202 Accepted: The request is being processed. No result is available yet. Continue polling.

HTTP 200 OK: Processing is complete. The response body contains a Parameters resource with the result. For example, here is a sample response from a successful request:

```
{
  "resourceType": "Parameters",
  "parameter": [{
    "name": "result",
    "valueCodeableConcept": {
      "coding": [{
        "system": "https://dsacms.github.io/cmmi-access-
model/CodeSystem/ACCESSAlignmentResultCS",
        "code": "aligned",
        "display": "Aligned"
      }],
      "text": "Patient is eligible and has been aligned so the
ACCESS participant can now begin providing services to the patient
under the model."
    }
```

```
      }]
    }
```

Clients should:

1. Extract `parameter[0].valueCodeableConcept` from the Parameters resource
2. Check `coding[0].code` against known alignment result codes
3. Display `text` element for user-friendly messaging
4. Take appropriate action based on the result code for the particular API. These actions are covered in the documentation for each API.

Error Handling

Clients should be prepared to handle the following conditions:

HTTP Status Code Errors:

- **400 Bad Request**: Malformed request - check Parameters structure and required fields
- **404 Not Found**: Submission ID not found - verify the submission ID is correct
- **500 Internal Server Error**: Server processing error - retry after delay or contact support
- **503 Service Unavailable**: Service temporarily unavailable - implement retry logic

Client-Side Errors:

- **Timeout**: No response received after maximum polling duration - log and notify user
- **Network Error**: Connection failure - implement retry logic with exponential backoff

Best Practice: Always check HTTP status codes before attempting to parse the response body. Only `HTTP 200` responses contain the result from the original request.

Security Considerations

- All API interactions **SHALL** use TLS 1.3
- Clients **SHALL** authenticate using approved OAuth 2.0
- Submission identifiers should be treated as sensitive information and protected accordingly
- Clients should implement appropriate audit logging for all requests
- The `Content-Location` HTTP header URL may contain sensitive identifiers and should be handled securely

Best Practices

1. **Idempotency**: Avoid submitting duplicate requests for the same patient
2. **Resource Validation**: Validate input resources against FHIR profiles before submission
3. **MBI Handling**: Ensure Medicare Beneficiary Identifiers (MBIs) are handled according to CMS data sharing policies
4. **Result Code Handling**: Implement robust parsing for all possible result codes
5. **Submission ID Management**: Store submission IDs securely and maintain their association with original requests

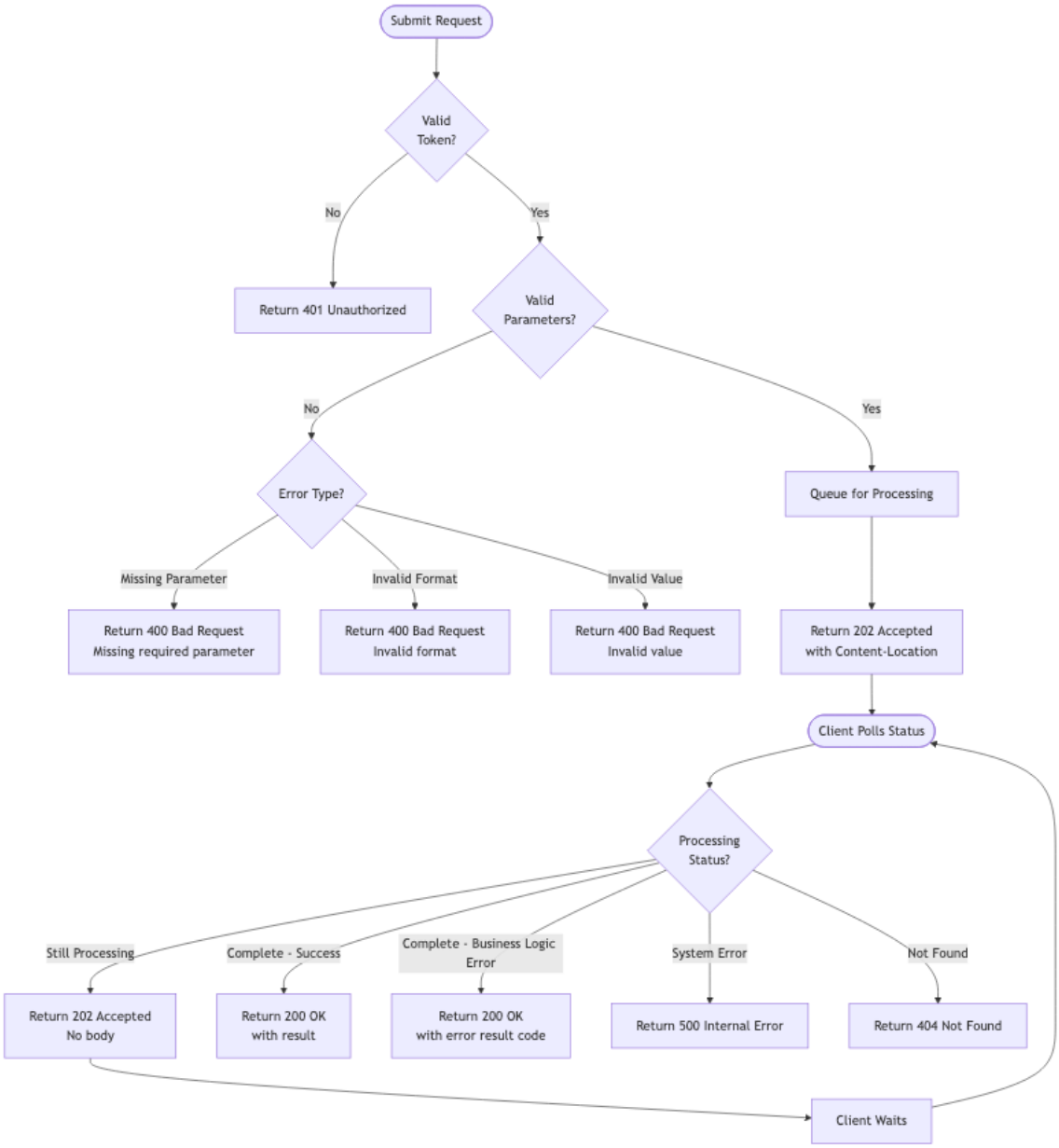
OperationOutcome

Errors are returned using FHIR OperationOutcome resources:

```
{
  "resourceType": "OperationOutcome",
  "issue": [
    {
      "severity": "error",
      "code": "invalid",
      "details": {
        "text": "Missing required parameter: participantID"
      }
    }
  ]
}
```

Error Handling Flow Diagram

The following diagram illustrates the complete error handling flow for ACCESS API interactions:



Key Decision Points:

1. **Authentication**: First check validates the OAuth token
2. **Parameter Validation**: Validates required parameters and their formats
3. **Asynchronous Processing**: Valid requests are queued and return `HTTP 202 (Accepted)`
4. **Status Polling**: Client repeatedly checks status until completion (returns something other than `HTTP 202`)
5. **Result Interpretation**:

- `HTTP 200` with success result code = operation succeeded
- `HTTP 200` with issue result code = business logic prevented operation (e.g., not-eligible, already-aligned)
- `HTTP 4xx/5xx` = technical error occurred

Common Error Scenarios

Submission Errors (400 Bad Request)

Missing Required Parameter:

```
POST https://[base]/access/Patient/$check-eligibility

HTTP/1.1 400 Bad Request
Content-Type: application/fhir+json

{
  "resourceType": "OperationOutcome",
  "issue": [
    {
      "severity": "error",
      "code": "required",
      "details": { "text": "Missing required parameter: patient" }
    }
  ]
}
```

Invalid MBI Format:

```
HTTP/1.1 400 Bad Request
Content-Type: application/fhir+json

{
  "resourceType": "OperationOutcome",
  "issue": [
    {
      "severity": "error",
      "code": "invalid",
      "details": { "text": "Invalid Medicare Beneficiary
Identifier (MBI) format" }
    }
  ]
}
```

Invalid Track Code:

```
HTTP/1.1 400 Bad Request
Content-Type: application/fhir+json

{
  "resourceType": "OperationOutcome",
  "issue": [
    {
      "severity": "error",
      "code": "code-invalid",
      "details": {
        "text": "Invalid track code. Must be one of: eCKM, CKM,
```

```
MSK, BH"
      }
    }
  ]
}
```

Malformed Parameters Resource:

```
HTTP/1.1 400 Bad Request
Content-Type: application/fhir+json

{
  "resourceType": "OperationOutcome",
  "issue": [
    {
      "severity": "error",
      "code": "structure",
      "details": {
        "text": "Parameters resource structure is invalid"
      },
      "diagnostics": "Expected Parameters.parameter array but
received malformed JSON"
    }
  ]
}
```

Invalid Participant ID (400):

```
HTTP/1.1 400 Forbidden
Content-Type: application/fhir+json

{
  "resourceType": "OperationOutcome",
  "issue": [
    {
      "severity": "error",
      "code": "invalid",
      "details": {
        "text": "Participant ID is not valid"
      }
    }
  ]
}
```

FHIR Validation Failure:

```
HTTP/1.1 400 Bad Request
Content-Type: application/fhir+json

{
  "resourceType": "OperationOutcome",
  "issue": [
    {
      "severity": "error",
      "code": "invalid",
      "details": {
        "text": "FHIR validation failed: Patient.identifier.system
is required when Patient.identifier.value is present"
```

```
      },
      "diagnostics": "The Patient resource does not conform to the
US Core Patient profile. MBI identifier must include both 'system'
(http://terminology.hl7.org/NamingSystem/cmsMBI) and 'value'
elements."
    }
  ]
}
```

Status Check Errors

Submission ID Not Found (404):

```
GET https://[base]/access/Patient/$submission-status/invalid-sub-
id

HTTP/1.1 404 Not Found
Content-Type: application/fhir+json

{
  "resourceType": "OperationOutcome",
  "issue": [
    {
      "severity": "error",
      "code": "not-found",
      "details": { "text": "Submission ID 'invalid-sub-id' not
found" }
    }
  ]
}
```

Authentication and Authorization Errors

Missing or Invalid Token (401):

```
HTTP/1.1 401 Unauthorized
WWW-Authenticate: Bearer error="invalid_token"
Content-Type: application/fhir+json

{
  "resourceType": "OperationOutcome",
  "issue": [
    {
      "severity": "error",
      "code": "security",
      "details": {
        "text": "Invalid or expired access token"
      }
    }
  ]
}
```

Insufficient Scope (403):

```
HTTP/1.1 403 Forbidden
Content-Type: application/fhir+json

{
  "resourceType": "OperationOutcome",
```

```
  "issue": [
    {
      "severity": "error",
      "code": "forbidden",
      "details": {
        "text": "Insufficient scope for requested operation."
      }
    }
  ]
}
```

Server Errors

Service Temporarily Unavailable (503):

```
HTTP/1.1 503 Service Unavailable
Retry-After: 60
Content-Type: application/fhir+json

{
  "resourceType": "OperationOutcome",
  "issue": [
    {
      "severity": "error",
      "code": "transient",
      "details": {
        "text": "Service temporarily unavailable. Please retry
after 60 seconds."
      }
    }
  ]
}
```

Internal Server Error (500):

```
HTTP/1.1 500 Internal Server Error
Content-Type: application/fhir+json

{
  "resourceType": "OperationOutcome",
  "issue": [
    {
      "severity": "error",
      "code": "exception",
      "details": {
        "text": "An unexpected error occurred processing your
request. Please contact support with submission ID if this
persists."
      }
    }
  ]
}
```

Example Status Check Request (Processing Complete - Aligned and Switch Approved):

```
GET https://[base]/access/Patient/$submission-status/sub-123456
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/fhir+json

{
  "resourceType": "Parameters",
  "parameter": [
    {
      "name": "result",
      "valueCodeableConcept": {
        "coding": [
          {
            "system": "https://dsacms.github.io/cmmi-access-
model/CodeSystem/ACCESSAlignmentResultCS",
            "code": "aligned-switch-approved",
            "display": "Aligned and switch approved"
          }
        ],
        "text": "The request to switch the patient's alignment
from a different participant after the 3-month lock in period is
accepted and the patient is considered switched and now re-
aligned."
      }
    }
  ]
}
```

There are other types of errors specific to each API that may be returned. Please see the section specific to that API in this manual to get more information and examples for those errors.

Standards Alignment

This asynchronous pattern aligns with:

- **FHIR Asynchronous Request Pattern**: Follows FHIR's general guidance for asynchronous operations using `HTTP 202 Accepted`
- **HTTP Standards**: Uses appropriate HTTP status codes (`202 Accepted`, `200 OK`, `4xx/5xx for errors`)
- **FHIR Parameters Resource**: Uses standardized Parameters resources for operation input and output
- **FHIR Operations Framework**: Implements custom operations with proper input/output handling
- **FHIR OperationOutcome**: Uses OperationOutcome for error responses following FHIR conventions
- **US Core Profiles**: Aligns with US Core Patient and Condition profiles for patient data

For additional information about FHIR patterns, see:

- FHIR Asynchronous Request Pattern (https://hl7.org/fhir/R4/async.html)
- FHIR Parameters Resource (https://hl7.org/fhir/R4/parameters.html)

- FHIR Operations Framework (https://hl7.org/fhir/R4/operations.html)
- FHIR OperationOutcome (https://hl7.org/fhir/R4/operationoutcome.html)
- US Core Implementation Guide (https://hl7.org/fhir/us/core/STU6.1/)

Implementation Notes and Edge Cases

Handling Network Interruptions

Scenario: Network failure occurs during polling

Implementation Guidance:

- Store submission IDs and associated request context before starting to poll
- Implement resume logic that can continue polling after network restoration
- Use exponential backoff when reconnecting after network failure
- Set reasonable timeout limits (5 minutes recommended) to avoid infinite polling loops

Duplicate Submission Prevention

Scenario: User clicks submit button multiple times or system retries unnecessarily

Implementation Guidance:

- Implement client-side button disabling after first click
- Store submission IDs locally to detect duplicates
- Check if a recent submission exists for the same patient/operation before submitting again
- Use idempotency tokens if retrying after uncertain failures

Token Expiration During Long Operations

Scenario: OAuth token expires while waiting for async operation to complete

Implementation Guidance:

- Refresh tokens proactively before expiration (5 minutes before recommended)
- Store token expiration time and check before each status poll
- Implement automatic token refresh in polling loop
- Handle `HTTP 401` responses by refreshing token and retrying once

Multiple Diagnoses Across Tracks

Scenario: Patient has conditions qualifying for multiple tracks

Implementation Guidance:

- Patient can be aligned to multiple tracks simultaneously*
- Each track requires a separate alignment request
- Track alignments are independent - success/failure in one does not affect others
- Data reporting is track-specific - submit separate bundles for each track
- Patient can be aligned to different participants or the same participant in different tracks

* **The one exception** to this rule applies to the eCKM and CKM tracks. Patients cannot be enrolled in both tracks at the same time. If a patient is already aligned to the eCKM track and elects to enroll in the CKM track, or vice versa, they can change tracks immediately. In this case the 3-month lock-in period does not apply.

Next Steps and support

- Review the Authentication guidance
- Consult API-specific guidance pages under the API Guidance menu
- Reference the Artifacts page in the ACCESS Model FHIR IG for technical specifications
- Contact the ACCESS Model support team if you have any questions (ACCESSModelTeam@cms.hhs.gov)

Authentication

Authentication Process for ACCESS APIs

This section describes the authentication and authorization process required to access all ACCESS Model API endpoints.

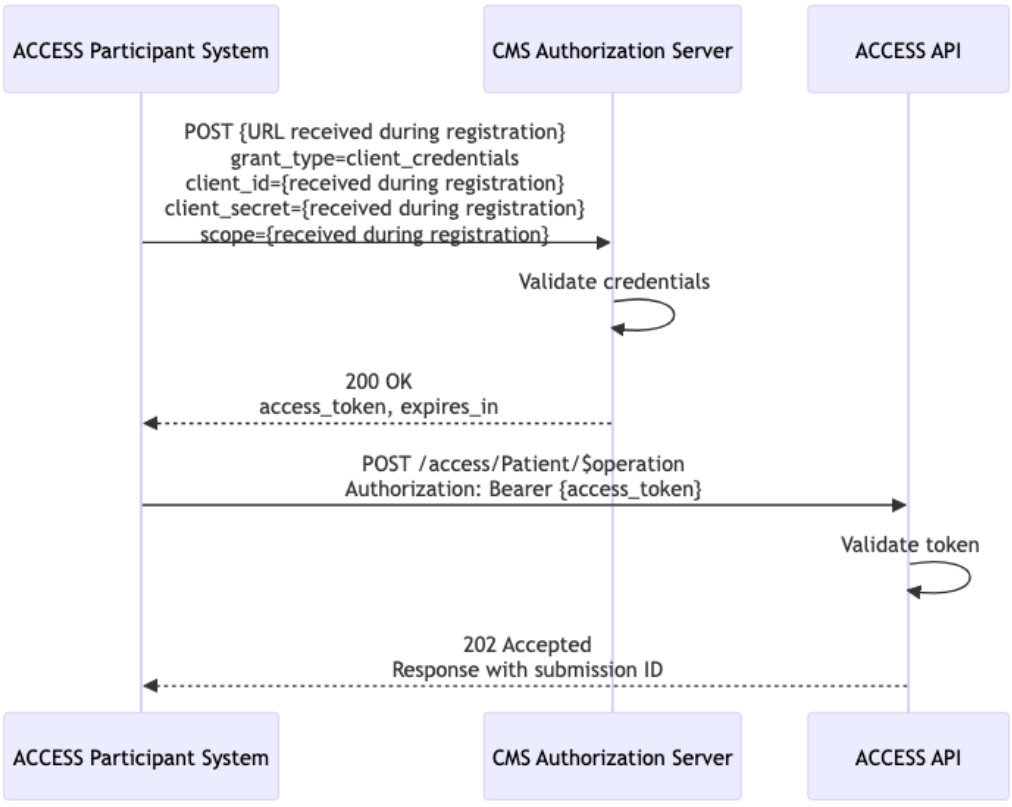
Overview

The ACCESS APIs use OAuth 2.0 for authentication and authorization. All ACCESS participants must complete a registration process and obtain appropriate credentials before accessing any API.

OAuth 2.0 Client Credentials Flow

The ACCESS APIs use the OAuth 2.0 Client Credentials grant type, which is appropriate for server-to-server authentication where the client (ACCESS participant system) is acting on its own behalf.

Authentication Flow



Step 1: Obtain Access Token

Token Endpoint: `POST {URL received during registration}`

Request Headers:

```
Content-Type: application/x-www-form-urlencoded
```

Request Body (form-encoded):

```
grant_type=client_credentials
client_id={received during registration}
client_secret={received during registration}
scope={received during registration}
```

Example cURL Request:

```
curl -X POST {URL received during registration} \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -d "grant_type=client_credentials" \
  -d "client_id={received during registration}" \
  -d "client_secret={received during registration}" \
  -d "scope={received during registration}"
```

Successful Response:

```
{
  "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "Bearer",
  "expires_in": 300,
  "scope": "{allowed scope}"
}
```

It is **strongly** recommended to get a new access token for each new request.

Step 2: Use Access Token in API Requests

Include the access token in the Authorization header of all API requests:

Request Headers:

```
Authorization: Bearer {access_token}
Content-Type: application/fhir+json
```

Example Request:

```
POST https://[base]/access/Patient/$check-eligibility
Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...
Content-Type: application/fhir+json

{
  "resourceType": "Parameters",
  "parameter": [...]
}
```

Token Management

Token Expiration

Access tokens have a limited lifetime (typically 5 minutes). Clients must:

1. **Monitor token expiration** - Track the `expires_in` value from the token response
2. **Refresh before expiration** - Request a new token before the current one expires

3. **Handle `HTTP 401` responses** - If a token expires mid-use, obtain a new token and retry the request

Token Caching Best Practices

- Cache tokens to avoid unnecessary authentication requests
- Request a new token 2 minutes before expiration to account for clock skew
- Implement thread-safe token management in multi-threaded applications
- Store credentials and tokens securely (never in source code or logs)

Example Token Management (Pseudocode)

```
class TokenManager:
    token = null
    expires_at = null

    function getValidToken():
        if token is null or currentTime() >= expires_at - 5 minutes:
            response = requestNewToken()
            token = response.access_token
            expires_at = currentTime() + response.expires_in
        return token

    function requestNewToken():
        return POST to auth_server with client_credentials
```

Required Scopes

The ACCESS APIs require the OAuth 2.0 scopes provided during the client registration process. Scopes may vary based on which APIs your organization has been approved to access.

Error Handling

Authentication Errors

Invalid Credentials (`HTTP 401 Unauthorized`):

```
{
  "error": "invalid_client",
  "error_description": "Client authentication failed"
}
```

Action: Verify your `client_id` and `client_secret` are correct.

Invalid Token (`HTTP 401 Unauthorized`):

```
HTTP/1.1 401 Unauthorized
WWW-Authenticate: Bearer error="invalid_token",
error_description="Token has expired"
```

Action: Request a new access token.

Insufficient Scope (`HTTP 403 Forbidden`):

```
{
  "resourceType": "OperationOutcome",
  "issue": [
    {
      "severity": "error",
      "code": "forbidden",
      "details": {
        "text": "Insufficient scope for requested operation"
      }
    }
  ]
}
```

Action: Ensure your token includes all required scopes for the API operation you are attempting to access.

Security Requirements

Transport Security

- **TLS 1.3 Required**: All communication must use TLS 1.3
- **Certificate Validation**: Clients must validate server certificates
- **No Insecure Protocols**: HTTP, TLS 1.0, TLS 1.1, TLS 1.2, and SSL are not permitted

Credential Security

- **Secure Storage**: Store client credentials in secure key management systems or vaults
- **No Code Storage**: Never store credentials in source code, configuration files, or version control
- **Environment Variables**: Use environment variables or secure configuration management
- **Rotation**: Implement credential rotation policies (recommended: every 90 days)
- **Audit Logging**: Log all authentication attempts (excluding sensitive credential data)

Token Security

- **Secure Transmission**: Only transmit tokens over TLS connections
- **Limited Lifetime**: Accept token expiration and request new tokens as needed
- **No Sharing**: Never share tokens between different applications or environments
- **Secure Storage**: If caching tokens, use encrypted storage
- **Logging**: Never log tokens in application logs or monitoring systems

Multi-API Access

A single set of credentials may provide access to multiple ACCESS APIs. The OAuth scopes in your access token will determine which operations you can perform.

Troubleshooting

Common Issues

Token Repeatedly Expires Mid-Request

- Check system clock synchronization

- Implement proactive token refresh (2 minutes before expiration)
- Verify network latency is not causing delays

Authentication Works in Test but Fails in Production

- Verify you are using production credentials (not test credentials)
- Check that production endpoint URLs are correct
- Confirm your organization has been approved for production access

`HTTP 403 Forbidden` Despite Valid Token

- Verify your scope includes the operation you are attempting
- Check that your participant ID is active
- Ensure you are using the correct API endpoints

Additional Resources

- OAuth 2.0 RFC 6749 (https://tools.ietf.org/html/rfc6749) - OAuth 2.0 specification
- OAuth 2.0 Client Credentials (https://tools.ietf.org/html/rfc6749#section-4.4) - Client credentials grant flow
- JWT RFC 7519 (https://tools.ietf.org/html/rfc7519) - JSON Web Token specification
- OAuth 2.0 Security Best Practices (https://datatracker.ietf.org/doc/html/draft-ietf-oauth-security-topics) - Security considerations

Eligibility API Guidance

How to Use the API

The ACCESS Eligibility API uses an asynchronous pattern for checking patient eligibility for the CMS ACCESS Model. The pattern uses two operations to support a submit-and-poll workflow:

1. **`$check-eligibility`**: Submits an eligibility check request
2. **`$submission-status`**: Polls for the status of a submitted request

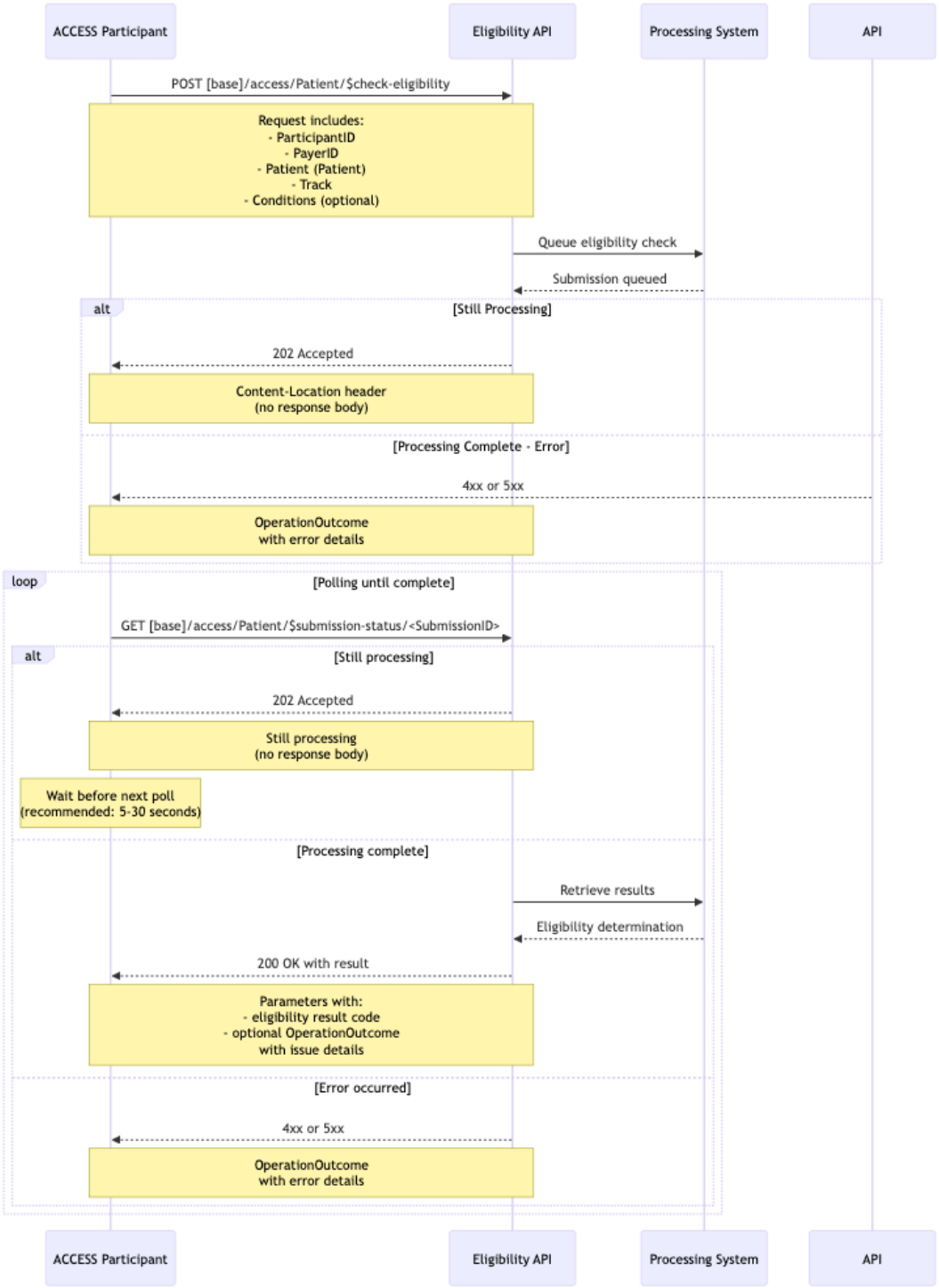
Asynchronous Interaction Pattern

The Eligibility API uses an asynchronous request-response pattern to accommodate the processing time required for eligibility determination. This pattern allows clients to submit eligibility check requests and subsequently poll for results without maintaining a persistent connection.

These operations can take some time to complete, making a synchronous request-response pattern impractical for real-world implementations.

Detailed Interaction Sequence

The following sequence diagram illustrates the complete interaction pattern:


ACCESS Participant
Eligibility API
Processing System
API
POST [base]/access/Patient/$check-eligibility
Request includes:
- ParticipantID
- PayerID
- Patient (Patient)
- Track
- Conditions (optional)
Queue eligibility check
Submission queued
alt
[Still Processing]
202 Accepted
Content-Location header
(no response body)
[Processing Complete - Error]
4xx or 5xx
OperationOutcome
with error details
loop
[Polling until complete]
GET [base]/access/Patient/$submission-status/<SubmissionID>
alt
[Still processing]
202 Accepted
Still processing
(no response body)
Wait before next poll
(recommended: 5-30 seconds)
[Processing complete]
Retrieve results
Eligibility determination
200 OK with result
Parameters with:
- eligibility result code
- optional OperationOutcome
with issue details
[Error occurred]
4xx or 5xx
OperationOutcome
with error details
ACCESS Participant
Eligibility API
Processing System
API

Operation Details

$check-eligibility Operation

The `$check-eligibility` operation initiates an asynchronous eligibility determination request.

Endpoint: `POST [base]/access/Patient/$check-eligibility`

Input Parameters:

- **`participantID`** (Identifier, required): The ACCESS ID for the submitting participant
- **`payerID`** (Identifier, required): The payer ID for the patient. Uses the X12 EDI Payer ID standard (`urn:oid:2.16.840.1.113883.3.221.5`) with CARIN Blue Button identifier typing (http://hl7.org/fhir/us/carin-bb/CodeSystem/C4BBIdentifierType). EDI Payer IDs are 5-character alphanumeric codes widely used in healthcare transactions (e.g., eligibility checks, claims, remittance).
- **`patient`** (Patient Resource, required): Patient information conforming to US Core Patient Profile. **SHOULD** contain the Medicare Beneficiary Identifier (MBI). To be eligible and align to a CMS ACCESS participant, a patient must be a Medicare beneficiary.
- **`track`** (CodeableConcept, required): The ACCESS Model track to check for eligibility
- **`conditions`** (Condition Resources, optional): Zero or more Condition resources conforming to the ACCESS Condition Profile describing the patient's health concerns. Implementers **SHOULD** use track-specific profiles (`ACCESSeCKMCondition`, `ACCESSCKMCondition`, `ACCESSMSKCondition`, `ACCESSBHCondition`) when submitting conditions for a known track. A condition, or diagnosis code, does not need to be included in the eligibility check for it to process. But, if the code is included, it should be an expected code for the track the patient is being considered for.

  NOTE: The IG will accept any patient identifier and payer identifier so that this IG can be used by other payers looking to align with the ACCESS Model. However, for a patient to be eligible for the CMS ACCESS Model, they need to have Medicare fee-for-service as their primary insurance and you must submit a valid MBI as the patient identifier.

Expected Response:

- **Status Code**: `202 Accepted`
- **HTTP Headers**: `Content-Location` header containing the URL to check submission status
- **Response Body**: Empty (no body while processing)

Example Request:

```
POST https://[base]/access/Patient/$check-eligibility
Content-Type: application/fhir+json

{
  "resourceType": "Parameters",
  "parameter": [
```

```
    {
      "name": "participantID",
      "valueIdentifier": {
        "system": "https://dsacms.github.io/cmmi-access-
model/participant-id",
        "value": "ACCESS1234"
      }
    },
    {
      "name": "payerID",
      "valueIdentifier": {
        "type": {
          "coding": [{
            "system": "http://hl7.org/fhir/us/carin-
bb/CodeSystem/C4BBIdentifierType",
            "code": "payerid",
            "display": "Payer ID"
          }]
        },
        "system": "urn:oid:2.16.840.1.113883.3.221.5",
        "value": "12345"
      }
    },
    {
      "name": "patient",
      "resource": {
        "resourceType": "Patient",
        "id": "example",
        "identifier": [
          {
            "type": {
              "coding": [
                {
                  "system":
"http://terminology.hl7.org/CodeSystem/v2-0203",
                  "code": "MC"
                }
              ]
            },
            "system":
"http://terminology.hl7.org/NamingSystem/cmsMBI",
            "value": "1234567890A"
          }
        ],
        "name": [
          {
            "family": "Doe",
            "given": ["John"]
          }
        ],
        "gender" : "male",
        "birthDate": "1950-01-01"
      }
    },
    {
```

```
      "name": "track",
      "valueCodeableConcept": {
        "coding": [{
          "system": "https://dsacms.github.io/cmmi-access-
model/CodeSystem/ACCESSTrackCS",
          "code": "CKM",
          "display": "Cardio-Kidney-Metabolic track"
        }]
      }
    },
    {
      "name": "condition",
      "resource": {
        "resourceType": "Condition",
        "code": {
          "coding": [
            {
              "system": "http://hl7.org/fhir/sid/icd-10-cm",
              "code": "E11.9",
              "display": "Type 2 diabetes mellitus without
complications"
            }
          ]
        },
        "subject": {
          "reference": "Patient/example"
        }
      }
    }
  ]
}
```

Example Response:

```
HTTP/1.1 202 Accepted
Content-Location: https://[base]/access/Patient/$submission-
status/sub-123456
```

(No response body - processing asynchronously)

$submission-status Operation

The `$submission-status` operation retrieves the current status of a previously submitted eligibility check request. This is a **read-only operation** that uses the `HTTP GET` method.

Understanding Response Behavior:

- **`HTTP 202 Accepted`**: Request is still being processed. Response has **no body**. Continue polling.
- **`HTTP 200 OK`**: Processing is complete. Response **always** contains a Parameters resource with the `result` parameter indicating the outcome.
- **`HTTP 4xx/5xx`**: An error occurred. Response contains an OperationOutcome with error details (no Parameters resource).

Eligibility Result Codes:
When processing is complete (`HTTP 200`), the `result` parameter uses codes from the **ACCESSEligibilityResultVS** value set:

- **`eligible`**: Patient is eligible for services under the model. This response does not mean the patient has been aligned. It simply means the patient is eligible to be aligned.
- **`eligible-pending-diagnosis`**: Patient is provisionally eligible for services under the model depending on the diagnosis.
- **`eligible-switch-participants`**: Patient is eligible to switch participants. This code indicates that the patient is already aligned to another participant, but they are outside of the 3-month lock-in period with that participant.
- **`not-eligible-not-medicare`**: The patient either is not enrolled in Medicare Part A and Part B or dual eligible for Medicare and Medicaid, or they do not have Medicare as their primary insurance, so they are not eligible for services under the ACCESS Model.
- **`not-eligible-services`**: The patient is receiving services (including receiving hospice services or dialysis for end stage renal disease (ESRD)) making them ineligible to be part of the ACCESS Model. Patients who are part of the Program of All-Inclusive Care for the Elderly (PACE) Program are also not eligible for the ACCESS Model.
- **`not-eligible-diagnoses`**: The patient does not have a treating diagnosis that qualifies them for services in the track indicated and therefore cannot get services under the ACCESS Model.
- **`not-eligible-control-group`**: The patient is technically eligible for the ACCESS Model, but based on the randomized control group algorithm, the patient has been placed in the control group for 12 months and therefore cannot be aligned for 12 months.
- **`not-eligible-already-aligned`**: The patient is technically eligible, but has already aligned to another participant within the past 3 months and is receiving services under the ACCESS Model in the same track. A patient can only be aligned to one participant in each track.

Example Responses:

Response:

```
HTTP/1.1 202 Accepted
```

(No response body - still processing)

Example Status Check Request (Processing Complete - Eligible):

```
GET https://[base]/access/Patient/$submission-status/sub-123456
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/fhir+json

{
  "resourceType": "Parameters",
```

```
  "parameter": [
    {
      "name": "result",
      "valueCodeableConcept": {
        "coding": [
          {
            "system": "https://dsacms.github.io/cmmi-access-
model/CodeSystem/ACCESSEligibilityResultCS",
            "code": "eligible",
            "display": "Eligible"
          }
        ],
        "text": "Patient is eligible for services under the model.
This response does not mean the patient has been aligned. It
simply means the patient is eligible to be aligned."
      }
    }
  ]
}
```

Example Status Check Request (Processing Complete - Not Eligible):

```
GET https://[base]/access/Patient/$submission-status/sub-123456
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/fhir+json

{
  "resourceType": "Parameters",
  "parameter": [
    {
      "name": "result",
      "valueCodeableConcept": {
        "coding": [
          {
            "system": "https://dsacms.github.io/cmmi-access-
model/CodeSystem/ACCESSEligibilityResultCS",
            "code": "not-eligible-control-group",
            "display": "Not eligible - assigned to Control Group"
          }
        ],
        "text": "The patient is technically eligible for the
ACCESS Model, but based on the randomized control group algorithm,
the patient has been placed in the control group for 12 months and
therefore cannot be aligned for 12 months."
      }
    }
  ]
}
```

Example Complete Workflow

The following example demonstrates a complete eligibility check workflow:

Step 1: Submit Eligibility Check

```
POST https://[base]/access/Patient/$check-eligibility
Content-Type: application/fhir+json

{
  "resourceType": "Parameters",
  "parameter": [
    {
      "name": "participantID",
      "valueIdentifier": {
        "system": "https://dsacms.github.io/cmmi-access-
model/participant-id",
        "value": "ACCESS1234"
      }
    },
    {
      "name": "payerID",
      "valueIdentifier": {
        "type": {
          "coding": [{
            "system": "http://hl7.org/fhir/us/carin-
bb/CodeSystem/C4BBIdentifierType",
            "code": "payerid",
            "display": "Payer ID"
          }]
        },
        "system": "urn:oid:2.16.840.1.113883.3.221.5",
        "value": "12345"
      }
    },
    {
      "name": "patient",
      "resource": {
        "resourceType": "Patient",
        "id": "example",
        "identifier": [
          {
            "type": {
              "coding": [
                {
                  "system":
"http://terminology.hl7.org/CodeSystem/v2-0203",
                  "code": "MC"
                }
              ]
            },
            "system":
"http://terminology.hl7.org/NamingSystem/cmsMBI",
            "value": "1234567890A"
          }
        ],
        "name": [
          {
            "family": "Doe",
            "given": ["John"]
```

```
            }
          ],
          "gender" : "male",
          "birthDate": "1950-01-01"
        }
      },
      {
        "name": "track",
        "valueCodeableConcept": {
          "coding": [
            {
              "system": "https://dsacms.github.io/cmmi-access-
model/CodeSystem/ACCESSTrackCS",
              "code": "CKM",
              "display": "Cardio-Kidney-Metabolic track"
            }
          ]
        }
      },
      {
        "name": "condition",
        "resource": {
          "resourceType": "Condition",
          "code": {
            "coding": [
              {
                "system": "http://hl7.org/fhir/sid/icd-10-cm",
                "code": "E11.9",
                "display": "Type 2 diabetes mellitus without
complications"
              }
            ]
          },
          "subject": {
            "reference": "Patient/example"
          }
        }
      }
    ]
}
```

Response:

```
HTTP/1.1 202 Accepted
Content-Location: https://[base]/access/Patient/$submission-
status/sub-123456
```

Step 2: Extract Submission URL and Wait

- Extract submission URL from Content-Location header
- Wait 5 seconds before first poll

Step 3: First Status Check (after 5 seconds)

```
GET https://[base]/access/Patient/$submission-status/sub-123456
```

Response:

```
HTTP/1.1 202 Accepted
```

(Still processing - no body)

Step 4: Second Status Check (after another 15 seconds)

```
GET https://[base]/access/Patient/$submission-status/sub-123456
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/fhir+json

{
  "resourceType": "Parameters",
  "parameter": [
    {
      "name": "result",
      "valueCodeableConcept": {
        "coding": [
          {
            "system": "https://dsacms.github.io/cmmi-access-
model/CodeSystem/ACCESSEligibilityResultCS",
            "code": "eligible",
            "display": "Eligible"
          }
        ],
        "text": "Patient is eligible for services under the model.
This response does not mean the patient has been aligned. It
simply means the patient is eligible to be aligned."
      }
    }
  ]
}
```

Step 5: Process Result

- Parse the result code: `"eligible"`
- Display the text to the user
- Proceed with alignment workflow

API-Specific Response Scenarios

For general response handling guidance and common scenarios (missing parameters, invalid tokens, server errors, etc.), see the Error Handling section in General Guidance.

The following responses are specific to the Eligibility API and are returned as part of normal processing (`HTTP 200` with specific result codes):

Not Eligible - Already Aligned

Scenario: Patient is already aligned to another provider in the same track.

```
GET https://[base]/access/Patient/$submission-status/sub-123456

HTTP/1.1 200 OK
Content-Type: application/fhir+json

{
  "resourceType": "Parameters",
  "parameter": [
    {
      "name": "result",
      "valueCodeableConcept": {
        "coding": [
          {
            "system": "https://dsacms.github.io/cmmi-access-
model/CodeSystem/ACCESSEligibilityResultCS",
            "code": "not-eligible-already-aligned",
            "display": "Not eligible - already aligned to another
participant in the track"
          }
        ],
        "text": "The patient is technically eligible, but is
already aligned to another participant and receiving services
under the ACCESS Model in the same track. A patient can only be
aligned to one participant in each track."
      }
    }
  ]
}
```

Not Eligible - Control Group

Scenario: Patient has been randomized into the control group.

```
GET https://[base]/access/Patient/$submission-status/sub-123456

HTTP/1.1 200 OK
Content-Type: application/fhir+json

{
  "resourceType": "Parameters",
  "parameter": [
    {
      "name": "result",
      "valueCodeableConcept": {
        "coding": [
          {
            "system": "https://dsacms.github.io/cmmi-access-
model/CodeSystem/ACCESSEligibilityResultCS",
            "code": "not-eligible-control-group",
            "display": "Not eligible - assigned to Control Group"
          }
        ],
        "text": "The patient is technically eligible for the
```

```
ACCESS Model, but based on the randomized control group algorithm,
the patient has been placed in the control group for 12 months and
therefore cannot be aligned for 12 months."
      }
    }
  ]
}
```

Not Eligible - No Qualifying Diagnosis

Scenario: Patient lacks the required diagnosis for the specified track.

```
GET https://[base]/access/Patient/$submission-status/sub-123456

HTTP/1.1 200 OK
Content-Type: application/fhir+json

{
  "resourceType": "Parameters",
  "parameter": [
    {
      "name": "result",
      "valueCodeableConcept": {
        "coding": [
          {
            "system": "https://dsacms.github.io/cmmi-access-
model/CodeSystem/ACCESSSEligibilityResultCS",
            "code": "not-eligible-diagnoses",
            "display": "Not eligible - no qualifying diagnosis"
          }
        ],
        "text": "The patient does not have a treating diagnosis
that qualifies them for services in the track indicated and
therefore cannot get services under the ACCESS Model."
      }
    }
  ]
}
```

Not Eligible - Receiving Exclusionary Services

Scenario: Patient is receiving services that prevent ACCESS participation.

```
GET https://[base]/access/Patient/$submission-status/sub-123456

HTTP/1.1 200 OK
Content-Type: application/fhir+json

{
  "resourceType": "Parameters",
  "parameter": [
    {
      "name": "result",
      "valueCodeableConcept": {
        "coding": [
```

```
          {
            "system": "https://dsacms.github.io/cmmi-access-
model/CodeSystem/ACCESSEligibilityResultCS",
            "code": "not-eligible-services",
            "display": "Not eligible - receiving services that
prevent eligibility"
          }
        ],
        "text": "The patient is receiving services (such as
hospice, end stage renal disease (ESRD), etc.) making them
ineligible to be part of the ACCESS Model."
      }
    }
  ]
}
```

Not Eligible - Not Medicare

Scenario: Patient is not receiving Medicare or Medicare is not primary insurance.

```
GET https://[base]/access/Patient/$submission-status/sub-123456

HTTP/1.1 200 OK
Content-Type: application/fhir+json

{
  "resourceType": "Parameters",
  "parameter": [
    {
      "name": "result",
      "valueCodeableConcept": {
        "coding": [
          {
            "system": "https://dsacms.github.io/cmmi-access-
model/CodeSystem/ACCESSEligibilityResultCS",
            "code": "not-eligible-not-medicare",
            "display": "Not eligible - not receiving Medicare"
          }
        ],
        "text": "The patient either is not enrolled in Medicare
Part A and Part B or dual eligible for Medicare and Medicaid, or
they do not have Medicare as their primary insurance, so they are
not eligible for services under the ACCESS Model."
      }
    }
  ]
}
```

Note: These are successful responses (`HTTP 200`) that contain business logic results. All of these scenarios represent valid processing outcomes for eligibility checks.

Alignment API Guidance

How to Use the API

The ACCESS Alignment API uses an asynchronous pattern for aligning patients to a track and participant in the CMS ACCESS Model. The pattern uses two operations to support a submit-and-poll workflow:

1. **`$align`**: Submits an alignment request
2. **`$submission-status`**: Polls for the status of a submitted request

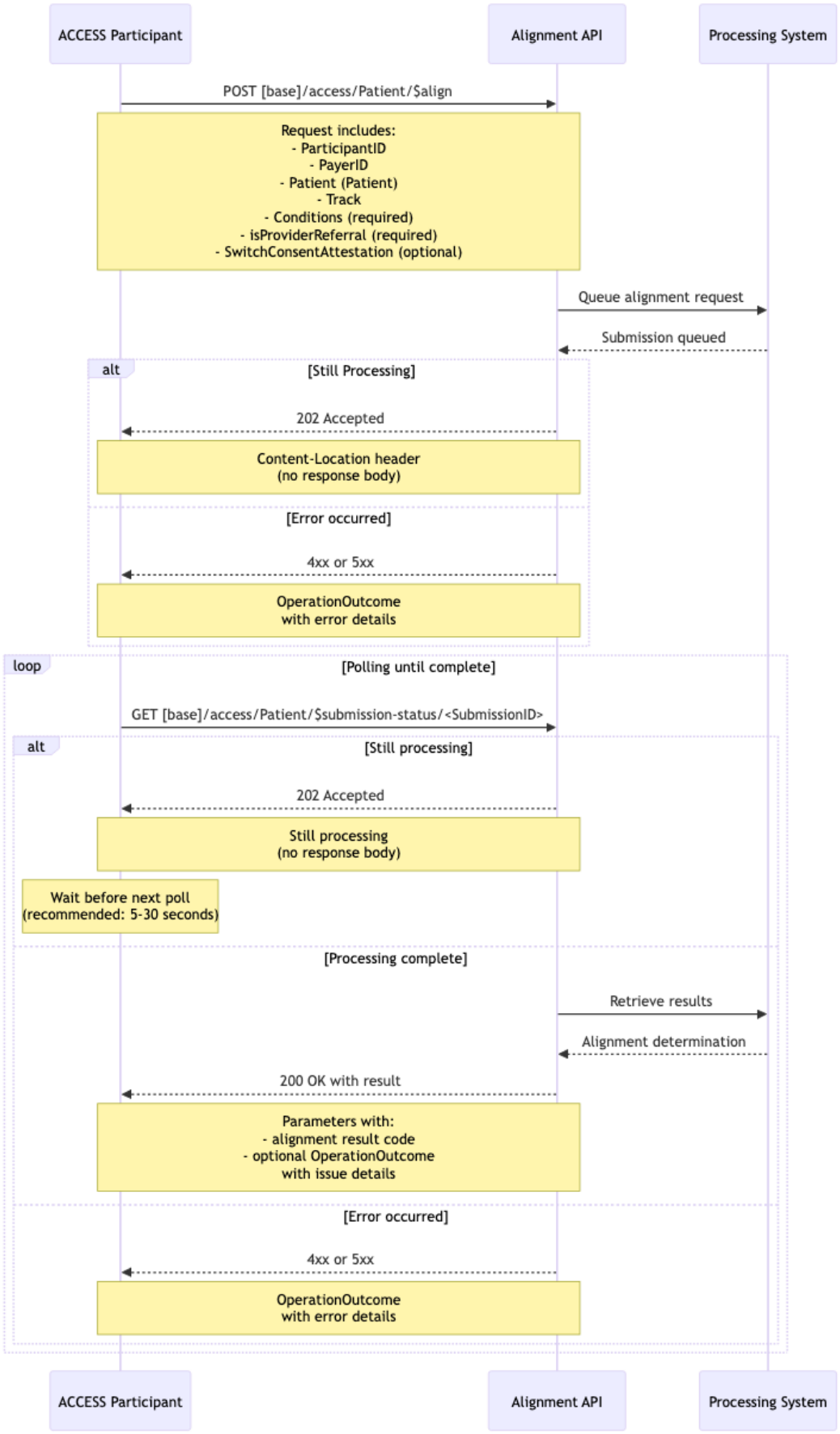
Asynchronous Interaction Pattern

The Alignment API uses an asynchronous request-response pattern to accommodate the processing time required for alignment determination. This pattern allows clients to submit alignment requests and subsequently poll for results without maintaining a persistent connection.

These operations can take some time to complete, making a synchronous request-response pattern impractical for real-world implementations.

Detailed Interaction Sequence

The following sequence diagram illustrates the complete interaction pattern:


ACCESS Participant
Alignment API
Processing System
POST [base]/access/Patient/$align
Request includes:
- ParticipantID
- PayerID
- Patient (Patient)
- Track
- Conditions (required)
- isProviderReferral (required)
- SwitchConsentAttestation (optional)
Queue alignment request
Submission queued
alt
[Still Processing]
202 Accepted
Content-Location header
(no response body)
[Error occurred]
4xx or 5xx
OperationOutcome
with error details
loop
[Polling until complete]
GET [base]/access/Patient/$submission-status/<SubmissionID>
alt
[Still processing]
202 Accepted
Still processing
(no response body)
Wait before next poll
(recommended: 5-30 seconds)
[Processing complete]
Retrieve results
Alignment determination
200 OK with result
Parameters with:
- alignment result code
- optional OperationOutcome
with issue details
[Error occurred]
4xx or 5xx
OperationOutcome
with error details
ACCESS Participant
Alignment API
Processing System

Operation Details

$align Operation

The `$align` operation initiates an asynchronous alignment request to align a patient to a specific track and participant.

Endpoint: `POST [base]/access/Patient/$align`

Input Parameters:

- **`participantID`** (Identifier, required): The ACCESS ID for the submitting participant
- **`payerID`** (Identifier, required): The payer ID for the patient. Uses the X12 EDI Payer ID standard (`urn:oid:2.16.840.1.113883.3.221.5`) with CARIN Blue Button identifier typing (http://hl7.org/fhir/us/carin-bb/CodeSystem/C4BBIdentifierType). EDI Payer IDs are 5-character alphanumeric codes widely used in healthcare transactions (e.g., eligibility checks, claims, remittance).
- **`patient`** (Patient Resource, required): Patient information conforming to US Core Patient Profile. **SHOULD** contain the Medicare Beneficiary Identifier (MBI)
- **`track`** (Value set, required): The requested track to align the patient
- **`conditions`** (Condition Resources, required): One or more Condition resources conforming to ACCESS Condition Profile describing the patient's health concerns. Implementers **SHALL** use track-specific profiles (`ACCESSeCKMCondition`, `ACCESSCKMCondition`, `ACCESSMSKCondition`, `ACCESSBHCondition`) when aligning to a specific track.
- **`isProviderReferral`** (boolean, required): Indicates whether the patient was referred to the ACCESS Model by a provider.
- **`switchConsentAttestation`** (boolean, optional): Boolean flag indicating that the patient has provided consent to switch from another ACCESS participant after the 3-month lock-in period. When `true`, this indicates the participant has obtained and documented the patient's consent to switch participants.

Expected Response:

- **Status Code**: `202 Accepted`
- **HTTP Headers**: `Content-Location` header containing the URL to check submission status
- **Response Body**: Empty (no body while processing)

Example Request:

```
POST https://[base]/access/Patient/$align
Content-Type: application/fhir+json

{
  "resourceType": "Parameters",
  "parameter": [
```

```
    {
      "name": "participantID",
      "valueIdentifier": {
        "system": "https://dsacms.github.io/cmmi-access-
model/participant-id",
        "value": "ACCESS1234"
      }
    },
    {
      "name": "payerID",
      "valueIdentifier": {
        "type": {
          "coding": [{
            "system": "http://hl7.org/fhir/us/carin-
bb/CodeSystem/C4BBIdentifierType",
            "code": "payerid",
            "display": "Payer ID"
          }]
        },
        "system": "urn:oid:2.16.840.1.113883.3.221.5",
        "value": "12345"
      }
    },
    {
      "name": "patient",
      "resource": {
        "resourceType": "Patient",
        "id": "example",
        "identifier": [
          {
            "type": {
              "coding": [
                {
                  "system":
"http://terminology.hl7.org/CodeSystem/v2-0203",
                  "code": "MC"
                }
              ]
            },
            "system":
"http://terminology.hl7.org/NamingSystem/cmsMBI",
            "value": "1234567890A"
          }
        ],
        "name": [
          {
            "family": "Doe",
            "given": ["John"]
          }
        ],
        "gender" : "male",
        "birthDate": "1950-01-01"
      }
    },
    {
```

```
      "name": "track",
      "valueCodeableConcept": {
        "coding": [
          {
            "system": "https://dsacms.github.io/cmmi-access-
model/CodeSystem/ACCESSTrackCS",
            "code": "CKM",
            "display": "Cardio-Kidney-Metabolic track"
          }
        ]
      }
    },
    {
      "name": "condition",
      "resource": {
        "resourceType": "Condition",
        "code": {
          "coding": [
            {
              "system": "http://hl7.org/fhir/sid/icd-10-cm",
              "code": "E11.9",
              "display": "Type 2 diabetes mellitus without
complications"
            }
          ]
        },
        "subject": {
          "reference": "Patient/example"
        }
      }
    },
    {
      "name": "isProviderReferral",
      "valueBoolean": true
    }
  ]
}
```

Example Response:

```
HTTP/1.1 202 Accepted
Content-Location: https://[base]/access/Patient/$submission-
status/sub-123456
```

(No response body - processing asynchronously)

$submission-status Operation

The `$submission-status` operation retrieves the current status of a previously submitted alignment request. This is a **read-only operation** that uses the HTTP GET method.

Endpoint: `GET [base]/access/Patient/$submission-status/<submission-id>`

Understanding Response Behavior:

- **`HTTP 202 Accepted`**: Request is still being processed asynchronously (no response body)
- **`HTTP 200 OK`**: Processing complete - response includes Parameters resource with alignment result
- **`HTTP 4xx/5xx`**: An error occurred. Response contains an OperationOutcome with error details (no Parameters resource)

Alignment Result Codes:
When processing is complete (`HTTP 200`), the `result` parameter uses codes from the **ACCESSAlignmentResultVS** value set:

- **`aligned`**: Patient is eligible and has been aligned so the participant can now begin providing services to the patient under the ACCESS Model.
- **`not-aligned-control-group`**: The patient is technically eligible, but based on the randomized control group algorithm, the patient has been placed in the control group for 12 months and therefore cannot be aligned for 12 months.
- **`not-aligned-already-aligned`**: The patient is technically eligible, but is already aligned to another participant and receiving services under the ACCESS Model in the same track. A patient can only be aligned to one participant in each track. If a switch consent attestation is submitted, but the patient is still within the 3-month lock-in period, this response will be received.
- **`not-aligned-not-medicare`**: The patient either is not enrolled in Medicare Part A and Part B or dual eligible for Medicare and Medicaid, or they do not have Medicare as their primary insurance, so they are not eligible for services under the ACCESS Model.
- **`not-aligned-services`**: The patient is receiving services (including receiving hospice services or dialysis for end stage renal disease (ESRD)) making them ineligible to be part of the ACCESS Model. Patients who are part of the Program of All-Inclusive Care for the Elderly (PACE) Program are also not eligible for the ACCESS Model.
- **`not-aligned-diagnoses`**: The patient does not have a treating diagnosis that qualifies them for service in the track indicated and therefore cannot get services under the ACCESS Model.
- **`aligned-switch-approved`**: The request to switch the patient's alignment from a different participant after the 3-month lock in period is accepted and the patient is considered switched and now re-aligned.

Parameter Encoding for GET:
The submission ID is passed as part of the URL path:

- Path: `GET [base]/access/Patient/$submission-status/sub-123456`

NOTE: If the $align operation returns `aligned` or `align-switch-approved`, the `submissionID` from that request is used to provide context for push notifications (e.g., emails) related to the alignment, connecting the participant, patient, and track. That `submissionID` is provided in the Beneficiary Alignment Report for future reference.

Example Status Check Request (Still Processing):

```
GET https://[base]/access/Patient/$submission-status/sub-123456
```

Response:

```
HTTP/1.1 202 Accepted
```

(No response body - still processing)

Example Status Check Request (Processing Complete - Aligned):

```
GET https://[base]/access/Patient/$submission-status/sub-123456
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/fhir+json

{
  "resourceType": "Parameters",
  "parameter": [
    {
      "name": "result",
      "valueCodeableConcept": {
        "coding": [
          {
            "system": "https://dsacms.github.io/cmmi-access-
model/CodeSystem/ACCESSAlignmentResultCS",
            "code": "aligned",
            "display": "Aligned"
          }
        ],
        "text": "Patient is eligible and has been aligned so the
participant can now begin providing services to the patient under
the ACCESS Model."
      }
    }
  ]
}
```

Example Status Check Request (Processing Complete - Not Aligned - No Qualifying Diagnosis):

```
GET https://[base]/access/Patient/$submission-status/sub-123456
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/fhir+json
```

```
{
  "resourceType": "Parameters",
  "parameter": [
    {
      "name": "result",
      "valueCodeableConcept": {
        "coding": [
          {
            "system": "https://dsacms.github.io/cmmi-access-
model/CodeSystem/ACCESSAlignmentResultCS",
            "code": "not-aligned-diagnoses",
            "display": "Not aligned - no qualifying diagnosis"
          }
        ],
        "text": "The patient does not have a treating diagnosis
that qualifies them for service in the track indicated and
therefore cannot get services under the ACCESS Model."
      }
    }
  ]
}
```


Example Status Check Request (Processing Complete - Not Aligned - Control Group):

```
GET https://[base]/access/Patient/$submission-status/sub-123456
```

Response:


```
HTTP/1.1 200 OK
Content-Type: application/fhir+json

{
  "resourceType": "Parameters",
  "parameter": [
    {
      "name": "result",
      "valueCodeableConcept": {
        "coding": [
          {
            "system": "https://dsacms.github.io/cmmi-access-
model/CodeSystem/ACCESSAlignmentResultCS",
            "code": "not-aligned-control-group",
            "display": "Not aligned - assigned to Control Group"
          }
        ],
        "text": "The patient is technically eligible, but based on
the randomized control group algorithm, the patient has been
placed in the control group for 12 months and therefore cannot be
aligned for 12 months."
      }
    }
  ]
}
```


Example Status Check Request (Processing Complete - Aligned and Switch Approved):

```
GET https://[base]/access/Patient/$submission-status/sub-123456
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/fhir+json

{
  "resourceType": "Parameters",
  "parameter": [
    {
      "name": "result",
      "valueCodeableConcept": {
        "coding": [
          {
            "system": "https://dsacms.github.io/cmmi-access-
model/CodeSystem/ACCESSAlignmentResultCS",
            "code": "aligned-switch-approved",
            "display": "Aligned and switch approved"
          }
        ],
        "text": "The request to switch the patient's alignment
from a different provider after the 3-month lock in period is
accepted and the patient is considered switched and now re-
aligned."
      }
    }
  ]
}
```

ACCESS Participant Switch Workflow

When a patient wishes to switch ACCESS participants after the 3-month lock-in period:

1. **Verify Eligibility to Switch**: First, use the Eligibility API to check if the patient is eligible to switch participants (result code `eligible-switch-participants`)
2. **Obtain Consent**: Obtain and document the patient's consent to switch participants
3. **Submit Switch Request**: Submit an alignment request with the `switchConsentAttestation` parameter set to `true`
4. **Process Result**: If approved, receive result code `aligned-switch-approved`

Important Notes:

- Patients must be aligned for at least 3 months before they can switch participants.
- If a switch request is submitted before the 3-month lock-in period expires, the result will be `not-aligned-already-aligned`
- The switch consent attestation must be documented to indicate the patient's explicit consent to change participants

Example Switch Request with Consent Attestation:

```
POST https://[base]/access/Patient/$align
Content-Type: application/fhir+json

{
  "resourceType": "Parameters",
  "parameter": [
```

```
    {
      "name": "participantID",
      "valueIdentifier": {
        "system": "https://dsacms.github.io/cmmi-access-
model/participant-id",
        "value": "ACCESS1234"
      }
    },
    {
      "name": "payerID",
      "valueIdentifier": {
        "type": {
          "coding": [{
            "system": "http://hl7.org/fhir/us/carin-
bb/CodeSystem/C4BBIdentifierType",
            "code": "payerid",
            "display": "Payer ID"
          }]
        },
        "system": "urn:oid:2.16.840.1.113883.3.221.5",
        "value": "12345"
      }
    },
    {
      "name": "patient",
      "resource": {
        "resourceType": "Patient",
        "id": "example",
        "identifier": [
          {
            "type": {
              "coding": [
                {
                  "system":
"http://terminology.hl7.org/CodeSystem/v2-0203",
                  "code": "MC"
                }
              ]
            },
            "system":
"http://terminology.hl7.org/NamingSystem/cmsMBI",
            "value": "1234567890A"
          }
        ],
        "name": [
          {
            "family": "Doe",
            "given": ["John"]
          }
        ],
        "gender": "male",
        "birthDate": "1950-01-01"
      }
    },
    {
```

```
      "name": "track",
      "valueCodeableConcept": {
        "coding": [
          {
            "system": "https://dsacms.github.io/cmmi-access-
model/CodeSystem/ACCESSTrackCS",
            "code": "CKM",
            "display": "Cardio-Kidney-Metabolic track"
          }
        ]
      }
    },
    {
      "name": "condition",
      "resource": {
        "resourceType": "Condition",
        "code": {
          "coding": [
            {
              "system": "http://hl7.org/fhir/sid/icd-10-cm",
              "code": "E11.9",
              "display": "Type 2 diabetes mellitus without
complications"
            }
          ]
        },
        "subject": {
          "reference": "Patient/example"
        }
      }
    },
    {
      "name": "isProviderReferral",
      "valueBoolean": false
    },
    {
      "name": "switchConsentAttestation",
      "valueBoolean": true
    }
  ]
}
```

Response if Switch Approved:

```
HTTP/1.1 202 Accepted
Content-Location: https://[base]/access/Patient/$submission-
status/sub-789012
```

Then polling `$submission-status` would eventually return:

```
{
  "resourceType": "Parameters",
  "parameter": [
    {
      "name": "result",
      "valueCodeableConcept": {
```

```
        "coding": [
          {
            "system": "https://dsacms.github.io/cmmi-access-
model/CodeSystem/ACCESSAlignmentResultCS",
            "code": "aligned-switch-approved",
            "display": "Aligned and switch approved"
          }
        ],
        "text": "The request to switch the patient's alignment
from a different participant after the 3-month lock in period is
accepted and the patient is considered switched and now re-
aligned."
      }
    }
  ]
}
```

Automatic Push Notifications

When an alignment is successful (result codes `aligned` or `aligned-switch-approved`), the system automatically begins sending push notifications via email about important upcoming events and deadlines. This helps ACCESS participants stay informed about critical dates and required actions.

Automatic Email Notifications are Sent for:

1. **Aligned** - Notification when patient has been aligned to a participant and ACCESS Model track
2. **Provider Lock-In Period Ending** - Notifications before the 3-month lock-in period ends
3. **Control Group Period Ending** - Notifications before the 12-month control group period ends
4. **Alignment Renewal Due** - Notifications when alignment renewal is approaching when a follow-on year is allowed
5. **Baseline Data Reporting Due** - Notifications when the deadline for baseline data submission is approaching
6. **Quarterly Data Reporting Due** - Notifications when the deadline for quarterly data submission is approaching
7. **End-of-Year Data Reporting Due** - Notifications when the deadline for end-of-year data submission is approaching
8. **Unalignment** - Notification for when a patient has been unaligned from the participant and/or track

What This Means for Implementers:

- **Email Addresses Configured During Onboarding**: Email addresses for receiving notifications are set up during participant registration
- **Automatic Notifications**: Emails are automatically sent when events occur—no additional configuration needed

- **Monitor Email**: Ensure registered email addresses are monitored regularly for time-sensitive notifications
- **Take Timely Action**: Respond to notifications before deadlines to maintain compliance

For complete details on email notification content, event types, and implementation guidance, see the Subscriptions section.

Best Practices

1. **Track Validation**: Verify that the track corresponds to a valid ACCESS track before submission
2. **Diagnosis Validation:** Ensure submitted conditions are relevant to the requested track
3. **Eligibility Pre-Check:** Consider checking eligibility via the Eligibility API before submitting alignment requests

Example Complete Workflow

The following example demonstrates a complete alignment workflow:

Step 1: Submit Alignment Request

```
POST https://[base]/access/Patient/$align
Content-Type: application/fhir+json

{
  "resourceType": "Parameters",
  "parameter": [
    {
      "name": "participantID",
      "valueIdentifier": {
        "system": "https://dsacms.github.io/cmmi-access-
model/participant-id",
        "value": "ACCESS1234"
      }
    },
    {
      "name": "payerID",
      "valueIdentifier": {
        "type": {
          "coding": [{
            "system": "http://hl7.org/fhir/us/carin-
bb/CodeSystem/C4BBIdentifierType",
            "code": "payerid",
            "display": "Payer ID"
          }]
        },
        "system": "urn:oid:2.16.840.1.113883.3.221.5",
        "value": "12345"
      }
    },
    {
      "name": "patient",
      "resource": {
        "resourceType": "Patient",
```

```
        "id": "example",
        "identifier": [
          {
            "type": {
              "coding": [
                {
                  "system":
"http://terminology.hl7.org/CodeSystem/v2-0203",
                  "code": "MC"
                }
              ]
            },
            "system":
"http://terminology.hl7.org/NamingSystem/cmsMBI",
            "value": "1234567890A"
          }
        ],
        "name": [
          {
            "family": "Doe",
            "given": ["John"]
          }
        ],
        "gender" : "male",
        "birthDate": "1950-01-01"
      }
    },
    {
      "name": "track",
      "valueCodeableConcept": {
        "coding": [
          {
            "system": "https://dsacms.github.io/cmmi-access-
model/CodeSystem/ACCESSTrackCS",
            "code": "CKM",
            "display": "Cardio-Kidney-Metabolic track"
          }
        ]
      }
    },
    {
      "name": "condition",
      "resource": {
        "resourceType": "Condition",
        "code": {
          "coding": [
            {
              "system": "http://hl7.org/fhir/sid/icd-10-cm",
              "code": "E11.9",
              "display": "Type 2 diabetes mellitus without
complications"
            }
          ]
        },
        "subject": {
```

```
            "reference": "Patient/example"
          }
        }
      },
      {
        "name": "condition",
        "resource": {
          "resourceType": "Condition",
          "code": {
            "coding": [
              {
                "system": "http://hl7.org/fhir/sid/icd-10-cm",
                "code": "I70.0",
                "display": "Atherosclerosis of aorta"
              }
            ]
          },
          "subject": {
            "reference": "Patient/example"
          }
        }
      },
      {
        "name": "isProviderReferral",
        "valueBoolean": true
      }
    ]
}
```

Response:

```
HTTP/1.1 202 Accepted
Content-Location: https://[base]/access/Patient/$submission-
status/sub-123456
```

Step 2: Extract Submission URL and Wait

- Extract submission URL from Content-Location header
- Wait 5 seconds before first poll

Step 3: First Status Check (after 5 seconds)

```
GET https://[base]/access/Patient/$submission-status/sub-123456
```

Response:

```
HTTP/1.1 202 Accepted
```

(Still processing - no body)

Step 4: Second Status Check (after another 15 seconds)

```
GET https://[base]/access/Patient/$submission-status/sub-123456
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/fhir+json

{
  "resourceType": "Parameters",
  "parameter": [
    {
      "name": "result",
      "valueCodeableConcept": {
        "coding": [
          {
            "system": "https://dsacms.github.io/cmmi-access-
model/CodeSystem/ACCESSAlignmentResultCS",
            "code": "aligned",
            "display": "Aligned"
          }
        ],
        "text": "Patient is eligible and has been aligned so the
participant can now begin providing services to the patient under
the ACCESS Model."
      }
    }
  ]
}
```

Step 5: Process Result

- Parse the result code: "`aligned`"
- Display the text to the user
- Begin providing services to the patient under the ACCESS Model

API-Specific Response Scenarios

For general response handling guidance and common scenarios (missing parameters, invalid tokens, server errors, etc.), see the Error Handling section in General Guidance.

The following responses are specific to the Alignment API and are returned as part of normal processing:

Not Aligned - Already Aligned (Lock-In Period Active)

Scenario: Patient is already aligned to another provider and is still within the 3-month lock-in period.

```
GET https://[base]/access/Patient/$submission-status/sub-123456
```

```
HTTP/1.1 200 OK
Content-Type: application/fhir+json

{
  "resourceType": "Parameters",
  "parameter": [
    {
      "name": "result",
      "valueCodeableConcept": {
        "coding": [
          {
            "system": "https://dsacms.github.io/cmmi-access-
model/CodeSystem/ACCESSAlignmentResultCS",
            "code": "not-aligned-already-aligned",
            "display": "Not aligned - already aligned to another
participant in the track"
          }
        ],
        "text": "The patient is technically eligible, but is
already aligned to another participant and receiving services
under the ACCESS Model in the same track. A patient can only be
aligned to one participant in each track. If a switch consent
attestation is submitted, but the patient is still within the 3-
month lock-in period, this response will be received."
      }
    }
  ]
}
```

Not Aligned - Missing Required Consent for Switch

Scenario: Patient is attempting to switch providers after lock-in period but consent attestation is missing.

```
HTTP/1.1 400 Bad Request
Content-Type: application/fhir+json

{
  "resourceType": "OperationOutcome",
  "issue": [
    {
      "severity": "error",
      "code": "required",
      "details": {
        "text": "Switch consent attestation is required when
patient is switching participants after lock-in period"
      },
      "diagnostics": "Patient is eligible to switch participants
but the 'switchConsentAttestation' parameter must be provided."
    }
  ]
}
```

Not Aligned - No Qualifying Diagnosis

Scenario: Patient does not have a diagnosis that qualifies for the requested track.

```
HTTP/1.1 200 OK
Content-Type: application/fhir+json

{
  "resourceType": "Parameters",
  "parameter": [
    {
      "name": "result",
"valueCodeableConcept": {
        "coding": [
          {
            "system": "https://dsacms.github.io/cmmi-access-
model/CodeSystem/ACCESSAlignmentResultCS",
            "code": "not-aligned-diagnoses",
            "display": "Not aligned - no qualifying diagnosis"
          }
        ],
        "text": "The patient does not have a treating diagnosis
that qualifies them for service in the track indicated and
therefore cannot get services under the ACCESS Model."
      }
    }
  ]
}
```

Missing Required Conditions Parameter

Scenario: Alignment request submitted without required conditions.

```
POST https://[base]/access/Patient/$align

HTTP/1.1 400 Bad Request
Content-Type: application/fhir+json

{
  "resourceType": "OperationOutcome",
  "issue": [
    {
      "severity": "error",
      "code": "required",
      "details": {
        "text": "Missing required parameter: conditions"
      },
      "diagnostics": "At least one Condition resource must be
provided for alignment requests. Conditions must conform to the US
Core Condition profile and include ICD-10-CM codes."
    }
  ]
}
```

Note: These scenarios represent a mix of business logic results (`HTTP 200` with specific result codes) and technical validation errors (`HTTP 400`). Always check the HTTP status code first to determine the type of response.

Implementation Notes and Edge Cases

Patient Switches Between Participants

Scenario: Patient aligned to ACCESS Participant A wants to switch to Participant B

Implementation Guidance:

1. Participant B checks eligibility (may return `not-eligible-already-aligned` if within 3-month lock-in period)
2. Wait until 3-month lock-in period expires
3. Patient provides consent to switch and Participant documents the consent
4. Participant B submits alignment with `switchConsentAttestation = true`.
5. If approved, patient is automatically unaligned from Participant A
6. Participant A receives unalignment notification automatically

Important Notes:

- Participant A does NOT need to manually unalign the patient
- Participant B handles the entire switch process
- System ensures only one active alignment per track
- Lock-in period prevents frequent participant switching

Unalignment API Guidance

How to Use the API

The ACCESS Unalignment API uses an asynchronous pattern for processing manual unalignment requests for patients in the ACCESS Model. The pattern uses two operations to support a submit-and-poll workflow:

1. **`$unalign`**: Submits an unalignment request
2. **`$submission-status`**: Polls for the status of a submitted request

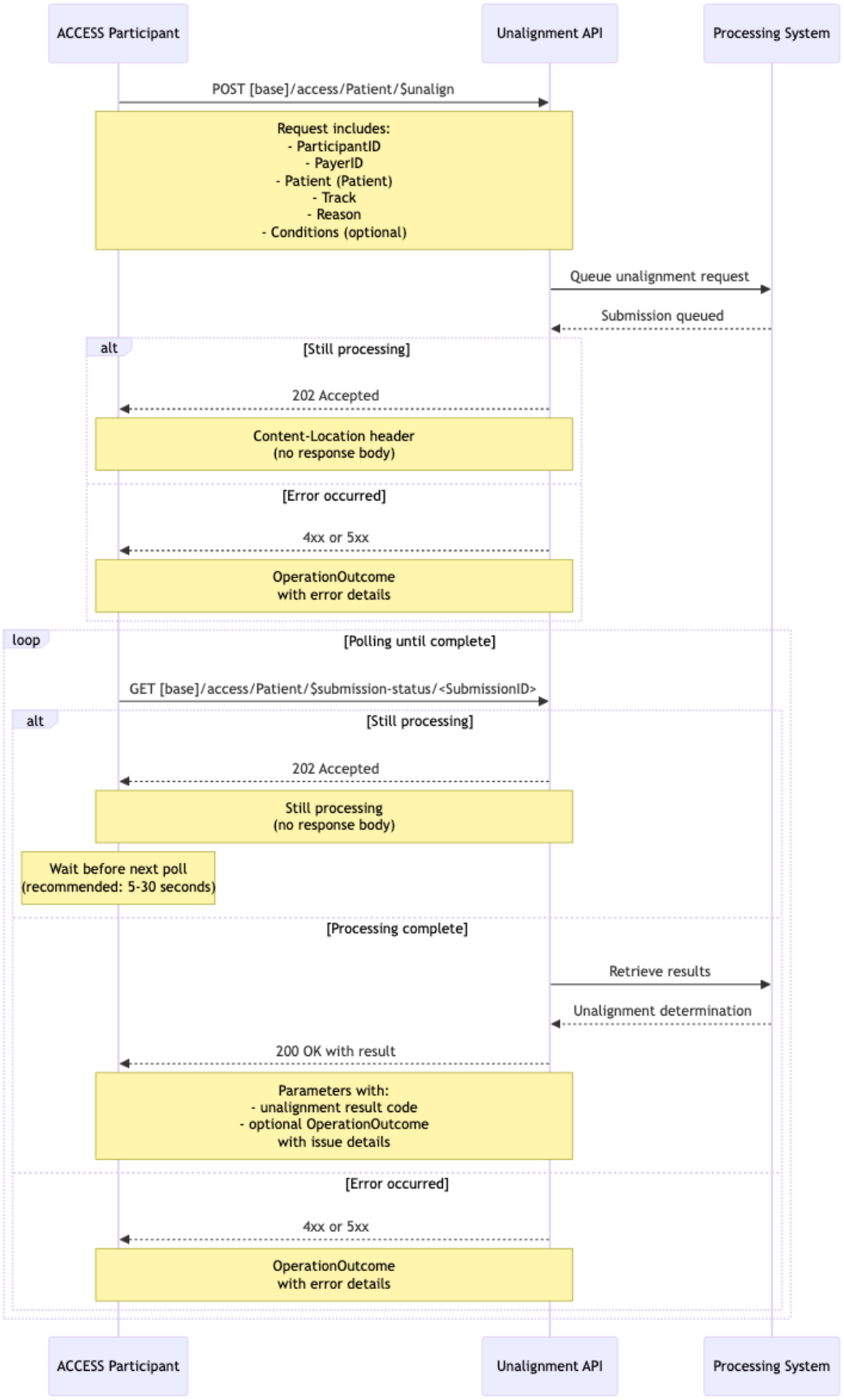
Asynchronous Interaction Pattern

The Unalignment API uses an asynchronous request-response pattern to accommodate the processing time required for unalignment determination. This pattern allows clients to submit unalignment requests and subsequently poll for results without maintaining a persistent connection.

These operations can take some time to complete, making a synchronous request-response pattern impractical for real-world implementations.

Detailed Interaction Sequence

The following sequence diagram illustrates the complete interaction pattern:


ACCESS Participant
Unalignment API
Processing System
POST [base]/access/Patient/$unalign
Request includes:
- ParticipantID
- PayerID
- Patient (Patient)
- Track
- Reason
- Conditions (optional)
Queue unalignment request
Submission queued
alt
[Still processing]
202 Accepted
Content-Location header
(no response body)
[Error occurred]
4xx or 5xx
OperationOutcome
with error details
loop
[Polling until complete]
GET [base]/access/Patient/$submission-status/<SubmissionID>
alt
[Still processing]
202 Accepted
Still processing
(no response body)
Wait before next poll
(recommended: 5-30 seconds)
[Processing complete]
Retrieve results
Unalignment determination
200 OK with result
Parameters with:
- unalignment result code
- optional OperationOutcome
with issue details
[Error occurred]
4xx or 5xx
OperationOutcome
with error details
ACCESS Participant
Unalignment API
Processing System

Operation Details

$unalign Operation

The `$unalign` operation initiates an asynchronous manual unalignment request.

Endpoint: `POST [base]/access/Patient/$unalign`

Input Parameters:

- **`participantID`** (Identifier, required): The ACCESS ID for the submitting participant
- **`payerID`** (Identifier, required): The payer ID for the patient. Uses the X12 EDI Payer ID standard (`urn:oid:2.16.840.1.113883.3.221.5`) with CARIN Blue Button identifier typing (http://hl7.org/fhir/us/carin-bb/CodeSystem/C4BBIdentifierType). EDI Payer IDs are 5-character alphanumeric codes widely used in healthcare transactions (e.g., eligibility checks, claims, remittance).
- **`patient`** (Patient Resource, required): Patient information conforming to US Core Patient Profile. **SHOULD** contain the Medicare Beneficiary Identifier (MBI)
- **`track`** (Identifier, required): The track from which the patient should be unaligned
- **`reason`** (CodeableConcept, required): The reason for the manual unalignment request. Value from **ACCESSUnalignmentReasonVS** value set
- **`conditions`** (Condition Resources, conditional): Zero or more Condition resources conforming to the ACCESS Condition Profile describing the patient's health concerns. **NOTE: This parameter is required** when `reason` is `no-longer-clinically-eligible` to document the disqualifying diagnosis (enforced by invariant `access-unalign-condition-required`)

Expected Response:

- **Status Code**: `202 Accepted`
- **HTTP Headers**: `Content-Location` header containing the URL to check submission status
- **Response Body**: Empty (no body while processing)

Example Request:

```
POST https://[base]/access/Patient/$unalign
Content-Type: application/fhir+json

{
  "resourceType": "Parameters",
  "parameter": [
    {
      "name": "participantID",
      "valueIdentifier": {
        "system": "https://dsacms.github.io/cmmi-access-
model/participant-id",
```

```
        "value": "ACCESS1234"
      }
    },
    {
      "name": "payerID",
      "valueIdentifier": {
        "type": {
          "coding": [{
            "system": "http://hl7.org/fhir/us/carin-
bb/CodeSystem/C4BBIdentifierType",
            "code": "payerid",
            "display": "Payer ID"
          }]
        },
        "system": "urn:oid:2.16.840.1.113883.3.221.5",
        "value": "12345"
      }
    },
    {
      "name": "patient",
      "resource": {
        "resourceType": "Patient",
        "identifier": [
          {
            "type": {
              "coding": [
                {
                  "system":
"http://terminology.hl7.org/CodeSystem/v2-0203",
                  "code": "MC"
                }
              ]
            },
            "system":
"http://terminology.hl7.org/NamingSystem/cmsMBI",
            "value": "1234567890A"
          }
        ],
        "name": [
          {
            "family": "Doe",
            "given": ["John"]
          }
        ],
        "gender" : "male",
        "birthDate": "1950-01-01"
      }
    },
    {
      "name": "track",
      "valueCodeableConcept": {
        "coding": [
          {
            "system": "https://dsacms.github.io/cmmi-access-
model/CodeSystem/ACCESSTrackCS",
```

```
            "code": "CKM",
            "display": "Cardio-Kidney-Metabolic track"
          }
        ]
      }
    },
    {
      "name": "reason",
      "valueCodeableConcept": {
        "coding": [
          {
            "system": "https://dsacms.github.io/cmmi-access-
model/CodeSystem/ACCESSUnalignmentReasonCS",
            "code": "loss-of-contact",
            "display": "Loss of contact"
          }
        ]
      }
    }
  ]
}
```

Example Response:

```
HTTP/1.1 202 Accepted
Content-Location: https://[base]/access/Patient/$submission-
status/sub-123456
```

(No response body - processing asynchronously)

Example Request (No Longer Clinically Eligible):

This example demonstrates unalignment when a patient has developed a new condition that makes them ineligible for the ACCESS Model. In this case, the patient was aligned to the CKM (Cardio-kidney-metabolic) track for diabetes management, but has now developed end-stage renal disease (ESRD) requiring dialysis, which makes them ineligible for the ACCESS Model per program requirements.

```
POST https://[base]/access/Patient/$unalign
Content-Type: application/fhir+json

{
  "resourceType": "Parameters",
  "parameter": [
    {
      "name": "participantID",
      "valueIdentifier": {
        "system": "https://dsacms.github.io/cmmi-access-
model/participant-id",
        "value": "ACCESS1234"
      }
    },
    {
```

```
      "name": "payerID",
      "valueIdentifier": {
        "type": {
          "coding": [{
            "system": "http://hl7.org/fhir/us/carin-
bb/CodeSystem/C4BBIdentifierType",
            "code": "payerid",
            "display": "Payer ID"
          }]
        },
        "system": "urn:oid:2.16.840.1.113883.3.221.5",
        "value": "12345"
      }
    },
    {
      "name": "patient",
      "resource": {
        "resourceType": "Patient",
        "id": "example",
        "identifier": [
          {
            "type": {
              "coding": [
                {
                  "system":
"http://terminology.hl7.org/CodeSystem/v2-0203",
                  "code": "MC"
                }
              ]
            },
            "system":
"http://terminology.hl7.org/NamingSystem/cmsMBI",
            "value": "9876543210B"
          }
        ],
        "name": [
          {
            "family": "Smith",
            "given": ["Jane"]
          }
        ],
        "gender": "female",
        "birthDate": "1955-06-15"
      }
    },
    {
      "name": "track",
      "valueCodeableConcept": {
        "coding": [
          {
            "system": "https://dsacms.github.io/cmmi-access-
model/CodeSystem/ACCESSTrackCS",
            "code": "CKM",
            "display": "Cardio-Kidney-Metabolic track"
          }
```

```
        ]
      }
    },
    {
      "name": "reason",
      "valueCodeableConcept": {
        "coding": [
          {
            "system": "https://dsacms.github.io/cmmi-access-
model/CodeSystem/ACCESSUnalignmentReasonCS",
            "code": "no-longer-clinically-eligible",
            "display": "No longer clinically eligible"
          }
        ],
        "text": "Patient has developed end-stage renal disease
(ESRD) requiring dialysis, which makes them ineligible for the
ACCESS Model."
      }
    },
    {
      "name": "condition",
      "resource": {
        "id" : "ESRDConditionExample",
        "meta" : {
          "profile" : [
            "https://dsacms.github.io/cmmi-access-
model/StructureDefinition/access-clinical-exclusion-condition"
          ]
        },
        "text" : {
          "status" : "generated",
          "div" : "<div xmlns=\"http://www.w3.org/1999/xhtml\">End
stage renal disease (ESRD)</div>"
        },
        "resourceType": "Condition",
        "clinicalStatus": {
          "coding": [
            {
              "system":
"http://terminology.hl7.org/CodeSystem/condition-clinical",
              "code": "active",
              "display": "Active"
            }
          ]
        },
        "verificationStatus": {
          "coding": [
            {
              "system":
"http://terminology.hl7.org/CodeSystem/condition-ver-status",
              "code": "confirmed",
              "display": "Confirmed"
            }
          ]
        },
```

```
        "category": [
          {
            "coding": [
              {
                "system":
"http://terminology.hl7.org/CodeSystem/condition-category",
                "code": "problem-list-item",
                "display": "Problem List Item"
              }
            ]
          }
        ],
        "code": {
          "coding": [
            {
              "system": "http://hl7.org/fhir/sid/icd-10-cm",
              "code": "N18.6",
              "display": "End stage renal disease"
            }
          ]
        },
        "subject": {
          "reference": "Patient/example"
        },
        "onsetDateTime": "2026-01-15"
      }
    }
  ]
}
```

Example Response:

```
HTTP/1.1 202 Accepted
Content-Location: https://[base]/access/Patient/$submission-
status/sub-789012
```

(No response body - processing asynchronously)

$submission-status Operation

The `$submission-status` operation retrieves the current status of a previously submitted unalignment request. This is a **read-only operation** that uses the HTTP GET method.

Endpoint: `GET [base]/access/Patient/$submission-status/<submission-id>`

Output Parameters (when complete):

- **`result`** (CodeableConcept, required): Result code with descriptive text indicating unalignment status. Value from **ACCESSUnalignmentResultVS** value set.

HTTP Status Codes:

- **`202 Accepted`**: Request is still being processed asynchronously (no response body)
- **`200 OK`**: Processing complete - response includes Parameters resource with unalignment result
- **`400 Bad Request`**: Invalid request (returns OperationOutcome)
- **`404 Not Found`**: Submission ID not found (returns OperationOutcome)
- **`500 Internal Server Error`**: Server error during processing (returns OperationOutcome)

Unalignment Result Codes:
When processing is complete (`HTTP 200`), the `result` parameter uses codes from the **ACCESSUnalignmentResultVS** value set:

- **`unaligned`**: The request to unalign the patient has been accepted and the patient has been successfully unaligned.
- **`unalignment-pending`**: Additional review is needed and you will receive further information once the manual review of the unalignment request is complete.
- **`patient-not-aligned`**: Patient is not currently aligned to this participant in the specified track and therefore cannot be unaligned.

Unalignment Reason Codes:
The `reason` parameter in the `$unalign` operation uses codes from the **ACCESSUnalignmentReasonVS** value set:

- **`geographic-relocated`**: Patient has relocated outside of the geographic area in which the aligned participant is licensed to provide services.
- **`loss-of-contact`**: Despite good faith efforts (defined as making three or more outreach attempts over 30 or more days), contact with the patient has been lost and therefore no longer able to engage with them.
- **`no-longer-clinically-eligible`**: Patient's conditions have changed such that they are no longer eligible for the ACCESS Model.
- **`patient-initiated:`** Patient no longer wants to participate in the ACCESS Model after the initial 90-day lock-in period.

Parameter Encoding for GET:
The submission ID is passed as part of the URL path:

- Path: `GET https://[base]/access/Patient/$submission-status/sub-123456`

Example Status Check Request (Still Processing):

```
GET https://[base]/access/Patient/$submission-status/sub-123456
```

Response:

```
HTTP/1.1 202 Accepted
```

(No response body - still processing)

Example Status Check Request (Processing Complete - Unaligned):

```
GET https://[base]/access/Patient/$submission-status/sub-123456
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/fhir+json

{
  "resourceType": "Parameters",
  "parameter": [
    {
      "name": "result",
      "valueCodeableConcept": {
        "coding": [
          {
            "system": "https://dsacms.github.io/cmmi-access-
model/CodeSystem/ACCESSUnalignmentResultCS",
            "code": "unaligned",
            "display": "Unaligned"
          }
        ],
        "text": "The request to unalign the patient has been
accepted and the patient has been successfully unaligned."
      }
    }
  ]
}
```

Example Status Check Request (Pending Manual Review):

```
GET https://[base]/access/Patient/$submission-status/sub-123456
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/fhir+json

{
  "resourceType": "Parameters",
  "parameter": [
    {
      "name": "result",
      "valueCodeableConcept": {
        "coding": [
          {
            "system": "https://dsacms.github.io/cmmi-access-
model/CodeSystem/ACCESSUnalignmentResultCS",
            "code": "unalignment-pending",
            "display": "Unalignment pending further review"
```

```
          }
        ],
        "text": "Additional review is needed and you will receive
further information once the manual review of the unalignment
request is complete."
      }
    }
  ]
}
```

Automatic Notification Cleanup

When a patient is successfully unaligned (result code `unaligned`), the system automatically stops sending email notifications about that patient. This ensures participants only receive notifications for currently aligned patients.

NOTE: Subscriptions will be maintained until all model obligations are completed for the patient. Some obligations will occur post unalignment.

Unalignment Notification Process:
When the `$unalign` operation completes successfully, the following occurs automatically:

1. **Unalignment Email Sent**: The system sends an email notification to the participant informing them that the patient has been unaligned and the alignment relationship has ended.
2. **Future Notifications Stopped**: The system automatically stops sending future email notifications about events for that patient (e.g., reporting deadlines, renewal reminders) after all model obligations are complete for the patient.

Important Notes:
- **Automatic Process**: Both the unalignment notification delivery and stopping of future notifications happen automatically as part of the unalignment workflow. No additional action is required by the ACCESS participant.
- **Notification Sent First**: An unalignment notification email is sent to inform the participant of the alignment termination before future notifications are stopped.
- **Prevents Future Notifications**: Stopping notifications ensures that participants do not receive future emails about patients who are no longer aligned.
- **Pending Unalignments**: If the unalignment result is `unalignment-pending` (requiring manual review), email notifications continue and no unalignment notification is sent until the unalignment is finalized with an `unaligned` result. Additional information can also be found in the Beneficiary Alignment Report for these cases.
- **Other Patients Unaffected**: Participant will continue to receive notifications for all other aligned patients.

Cross-Reference:
For complete details on the email notifications that are sent during alignment, see the Subscriptions section and the Alignment API documentation.

Example Complete Workflow

The following example demonstrates a complete unalignment request workflow:

Step 1: Submit Unalignment Request

```
POST https://[base]/access/Patient/$unalign
Content-Type: application/fhir+json

{
  "resourceType": "Parameters",
  "parameter": [
    {
      "name": "participantID",
      "valueIdentifier": {
        "system": "https://dsacms.github.io/cmmi-access-
model/participant-id",
        "value": "ACCESS1234"
      }
    },
    {
      "name": "payerID",
      "valueIdentifier": {
        "type": {
          "coding": [{
            "system": "http://hl7.org/fhir/us/carin-
bb/CodeSystem/C4BBIdentifierType",
            "code": "payerid",
            "display": "Payer ID"
          }]
        },
        "system": "urn:oid:2.16.840.1.113883.3.221.5",
        "value": "12345"
      }
    },
    {
      "name": "patient",
      "resource": {
        "resourceType": "Patient",
        "identifier": [
          {
            "type": {
              "coding": [
                {
                  "system":
"http://terminology.hl7.org/CodeSystem/v2-0203",
                  "code": "MC"
                }
              ]
            },
            "system":
"http://terminology.hl7.org/NamingSystem/cmsMBI",
            "value": "1234567890A"
```

```
          }
        ],
        "name": [
          {
            "family": "Doe",
            "given": ["John"]
          }
        ],
        "gender" : "male",
        "birthDate": "1950-01-01"
      }
    },
    {
      "name": "track",
      "valueCodeableConcept": {
        "coding": [
          {
            "system": "https://dsacms.github.io/cmmi-access-
model/CodeSystem/ACCESSTrackCS",
            "code": "CKM",
            "display": "Cardio-Kidney-Metabolic track"
          }
        ]
      }
    },
    {
      "name": "reason",
      "valueCodeableConcept": {
        "coding": [
          {
            "system": "https://dsacms.github.io/cmmi-access-
model/CodeSystem/ACCESSUnalignmentReasonCS",
            "code": "geographic-relocated",
            "display": "Geographic relocated"
          }
        ],
        "text": "Patient has relocated outside of the geographic
area in which the aligned participant is licensed to provide
services."
      }
    }
  ]
}
```

Response:

```
HTTP/1.1 202 Accepted
Content-Location: https://[base]/access/Patient/$submission-
status/sub-123456
```

Step 2: Extract Submission URL and Wait

- Extract submission URL "sub-123456" from Content-Location header

- Wait 5 seconds before first poll

Step 3: First Status Check (after 5 seconds)

```
GET https://[base]/access/Patient/$submission-status/sub-123456
```

Response:

```
HTTP/1.1 202 Accepted
```

(Still processing - no body)

Step 4: Second Status Check (after another 15 seconds)

```
GET https://[base]/access/Patient/$submission-status/sub-123456
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/fhir+json

{
  "resourceType": "Parameters",
  "parameter": [
    {
      "name": "result",
      "valueCodeableConcept": {
        "coding": [
          {
            "system": "https://dsacms.github.io/cmmi-access-
model/CodeSystem/ACCESSUnalignmentResultCS",
            "code": "unaligned",
            "display": "Unaligned"
          }
        ],
        "text": "The request to unalign the patient has been
accepted and the patient has been successfully unaligned."
      }
    }
  ]
}
```

Step 5: Process Result

- Parse the result code: "`unaligned`"
- Display the text to the user
- Complete unalignment workflow and update internal systems

API-Specific Response Scenarios

For general response handling guidance and common scenarios (missing parameters, invalid tokens, server errors, etc.), see the Error Handling section in General Guidance.

The following responses are specific to the Unalignment API:

Not Unaligned - Patient Not Currently Aligned

Scenario: Attempting to unalign a patient who is not currently aligned to this ACCESS participant.

```
GET https://[base]/access/Patient/$submission-status/sub-123456

HTTP/1.1 200 OK
Content-Type: application/fhir+json

{
  "resourceType": "Parameters",
  "parameter": [
    {
      "name": "result",
      "valueCodeableConcept": {
        "coding": [
          {
            "system": "https://dsacms.github.io/cmmi-access-
model/CodeSystem/ACCESSUnalignmentResultCS",
            "code": "patient-not-aligned",
            "display": "Patient not aligned"
        ],
        "text": "Patient is not currently aligned to this
participant in the specified track and therefore cannot be
unaligned."
      }
    }
  ]
}
```

Invalid Unalignment Reason

Scenario: Unalignment reason code is not recognized or invalid for the situation.

```
HTTP/1.1 400 Bad Request
Content-Type: application/fhir+json

{
  "resourceType": "OperationOutcome",
  "issue": [
    {
      "severity": "error",
      "code": "code-invalid",
      "details": {
        "text": "Invalid unalignment reason code"
      },
      "diagnostics": "The reason code 'invalid-reason' is not a
valid value from the ACCESSUnalignmentReasonVS value set. Valid
reasons include: geographic-relocated, loss-of-contact, no-longer-
```

```
clinically-eligible, patient-initiated."
      }
    }
  ]
}
```

Missing Required Reason Parameter

Scenario: Unalignment request submitted without the required reason parameter.

```
POST https://[base]/access/Patient/$unalign

HTTP/1.1 400 Bad Request
Content-Type: application/fhir+json

{
  "resourceType": "OperationOutcome",
  "issue": [
    {
      "severity": "error",
      "code": "required",
      "details": {
        "text": "Missing required parameter: reason"
      },
      "diagnostics": "The 'reason' parameter is required for all
unalignment requests and must contain a CodeableConcept from the
ACCESSUnalignmentReasonVS value set."
    }
  ]
}
```

Note: These scenarios represent a mix of business logic results (`HTTP 200` with specific result codes) and technical validation errors (`HTTP 400`). Always check the HTTP status code first to determine the type of response.

Subscriptions Guidance

The ACCESS Model provides automatic email notifications for important events during the patient ACCESS Model lifecycle.

Overview

Upon successful patient alignment, participants are automatically enrolled to receive email notifications about upcoming deadlines and events that require action or awareness. These email notifications are automatically stopped when a patient is unaligned, either manually or through the ACCESS Model system.

Event Types

The following event types trigger email notifications to ACCESS participants:

Event Name	Event Code	Description	When Triggered
Provider Lock-In Period Ending	provider-lock-ending	The 3-month provider lock-in period is ending soon, allowing the patient to switch to a different provider if desired	Before the lock-in period ends
Control Group Period Ending	control-group-ending	The 12-month control group assignment period is ending soon, allowing the patient to reconsidered for alignment in the ACCESS Model	Before control group period ends
Alignment Renewal Due	alignment-renewal	The initial alignment period is ending and renewal action is required to continue services under the ACCESS Model	Before initial alignment period expires
Baseline Reporting Due	baseline-reporting-window	The baseline data report is due soon. If baseline reporting is not received within 60 days of alignment, the patient will be unaligned.	When baseline reporting deadline is approaching
Quarterly Reporting Due	quarterly-reporting-window	The quarterly data report is due soon. Quarterly reporting must be submitted 70 to 110 days after the prior data submission.	When quarterly reporting window opens

Event Name	Event Code	Description	When Triggered
End-of-Year Reporting Due	`eoy-reporting-window`	The end-of-year data report is due soon. End-of-year reporting must be submitted no later than 425 days from the date of alignment (365 days plus an additional 60 days).	When end-of-year reporting window opens
Unalignment Notice	`unalignment`	The patient has been unaligned from the ACCESS Model due to a change in eligibility status.	When patient is unaligned by the ACCESS Model system

Email Notification Enrollment

Email notifications are automatically configured during the participant onboarding process. The email addresses registered during onboarding will receive notifications for all relevant events related to aligned patients.

Key Points:

- Email addresses are configured during participant onboarding
- Notifications are sent automatically for all events
- No additional configuration needed during alignment

What Happens After Alignment:

Upon successful alignment (`HTTP 200` with result code `aligned` or `aligned-switch-approved`), the system automatically:

1. Begins monitoring for upcoming events that require notification
2. Sends email notifications to registered addresses when events occur
3. Continues notifications throughout the alignment period
4. Stops notifications when the patient is unaligned

You do not need to manually configure or manage these notifications - they are handled automatically by the system based on your registered email addresses.

Email Notification Content

Email notifications include:

- **Event Type**: Clear description of what is happening (e.g., "Participant Lock-In Period Ending")
- **Submission ID**: Submission ID from the successful alignment for the patient, provided in the Beneficiary Alignment Report

- **Participant ID**: Your ACCESS participant identifier
- **Deadline Date**: When action is required or event occurs
- **Action Required**: Specific steps you need to take
- **Additional Information**: Relevant details and guidance

Example Email Notification

```
Subject: ACCESS Model Notification - Participant Lock-In Period
Ending

Dear ACCESS Participant,

Event: Participant Lock-In Period Ending
Submission ID: sub-123456
Participant ID: ACCESS1234
Deadline Date: May 1, 2026

Action Required:
The 3-month participant lock-in period for this patient ends on
May 1, 2026.
After this date, the patient may switch to another ACCESS
participant if they choose.

Additional Information:
Please contact the patient to discuss their care options and
ensure they are aware of this upcoming change.

If you have questions, please contact ACCESS Model support at
ACCESSModelTeam@cms.hhs.gov.
```

Notification Lifecycle

Enrollment

- Automatic upon successful alignment
- Uses email addresses from participant onboarding
- Notifications sent for all relevant events

Active Period

- Remains active for duration of alignment
- Triggers email notifications at appropriate times

Cancellation

- Automatic upon successful unalignment
- No further notifications sent for that patient
- Participant continues to receive notifications for other aligned patients

Implementation Guidance

Receiving Notifications

Your email system should:

- Accept emails from ACCESS Model notification service
- Configure spam filters to allow ACCESS emails
- Set up forwarding rules if needed for team distribution
- Monitor inbox regularly for time-sensitive notifications

Processing Notifications

When you receive a notification email:

1. **Review Event Details** - Understand what is happening
2. **Check Deadline** - Note when action is required
3. **Update Internal Systems** - Record event in your system
4. **Take Appropriate Action** - Follow up with patient if needed
5. **Complete Required Actions** - Perform required tasks before deadline

Security Considerations

- Email notifications contain limited patient information for privacy
- Use secure email systems with encryption
- Train staff on handling Personally Identifiable Information (PII) and/or Protected Health Information (PHI) in emails
- Follow your organization's email security policies
- Do not forward notification emails outside your organization

Related Artifacts

- `ACCESSEventTypeCS` CodeSystem
- `ACCESSEventTypeVS` ValueSet

Testing Guide

This page provides detailed testing guidance to ensure implementations conform to ACCESS Model requirements before production deployment.

Testing Checklist

Before deploying to production, implementers should complete the following validation steps:

FHIR Conformance Validation:

- All resource instances validate against their declared profiles
- All Must Support elements are properly populated when data is available
- All required elements are present in submissions
- Value set bindings use codes from the correct code systems
- Patient resources include valid MBI in correct format

Operation Testing:

- All four operations (`$check-eligibility`, `$align`, `$unalign`, `$report-data`) successfully submit requests
- `Content-Location`n header is correctly parsed and used for polling
- `$submission-status` correctly returns `HTTP 202 Accepted` while processing
- `$submission-status` correctly returns `HTTP 200 OK` with result when complete
- All result codes are correctly handled

Authentication & Security:

- OAuth 2.0 token acquisition succeeds
- Token is correctly included in Authorization header
- Token refresh works before expiration
- TLS 1.3 is enforced
- MBI and PII/PHI are encrypted in transit and storage

Error Handling:

- `HTTP 400` errors are handled without retry
- `HTTP 401` errors trigger token refresh
- `HTTP 500/503` errors implement exponential backoff retry
- Network errors implement retry logic
- Timeout scenarios are handled gracefully

Asynchronous Pattern:

- Polling implements appropriate wait intervals (5-30 seconds)
- Polling includes timeout mechanism
- Submission IDs are stored and associated with original requests
- Polling continues after network interruption

Test Scenarios by API

Eligibility API Test Scenarios

Scenario	Input	Expected Result	Validation
Eligible Patient	Valid patient with qualifying diagnosis	HTTP 200, result "eligible"	Can proceed to alignment
Control Group	Patient randomized to control	HTTP 200, result "not-eligible-control-group"	12-month wait documented
Already Aligned	Patient aligned to another provider	HTTP 200, result "not-eligible-already-aligned"	Cannot align
Missing Parameter	Request without patient	HTTP 400, OperationOutcome	Error identifies missing parameter
Invalid Track	Track code not in value set	HTTP 400, OperationOutcome	Error identifies invalid track

Alignment API Test Scenarios

Scenario	Input	Expected Result	Validation
Successful Alignment	Eligible patient with conditions	HTTP 200, result "aligned"	Subscriptions created
Provider Switch	Alignment with switch consent after 3 months	HTTP 200, result "aligned-switch-approved"	Previous subscriptions cancelled
Lock-In Active	Switch attempt within 3 months	HTTP 200, result "not-aligned-already-aligned"	Switch rejected
Missing Conditions	Alignment without conditions	HTTP 400, OperationOutcome	Error identifies missing conditions
No Qualifying Diagnosis	Conditions don't match track	HTTP 200, result "not-aligned-diagnoses"	Specific requirements noted

Unalignment API Test Scenarios

Scenario	Input	Expected Result	Validation
Successful Unalignment	Valid reason (geographic-relocated)	HTTP 200, result "unaligned"	Subscriptions cancelled
Manual Review	Unalignment requiring review	HTTP 200, result "unalignment-pending"	Subscriptions remain active
Not Aligned	Patient not currently aligned	HTTP 200, result "not-unaligned-not-aligned"	Cannot unalign

Missing Reason	Unalignment without reason	HTTP 400, OperationOutcome	Error identifies missing reason

Validation Tools and Resources

FHIR Validation:

- Online validator: https://validator.fhir.org/